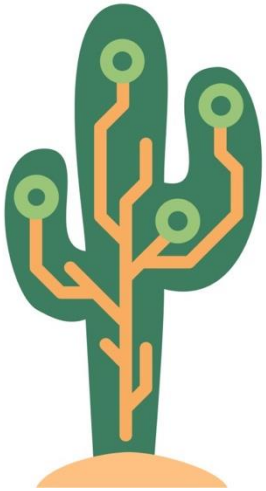
Ira A. Fulton Schools of Engineering
Arizona State University

Advanced Hardware and Systems for Machine Learning

Lecture 6: DNN Kernel Computation



Jeff (Jun) Zhang
Assistant Professor
School of ECEE, ASU
jeffzhang@asu.edu



Deep Learning is Everywhere







Efficient AI in the Wild

FSD (Supervised) v14.3.8

Full Self-Driving (Supervised) v14.3.8 includes:

- Upgraded the Reinforcement Learning (RL) stage of training the FSD neural network, resulting in improvements in a wide variety of driving scenarios.
- Upgraded the neural network vision encoder, improving understanding in rare and low-visibillty scenarios, strengthening 3D geometry understanding, and expanding traffic sign understanding.
- Rewrote the AI compiler and runtime from the ground up with MLIR, resulting in 20% faster reaction time and improving model iteration speed.
- Mitigated unnecessary lane biasing and minor tailgating behaviors.
- Increased decisiveness of parking spot selection and maneuvering.
- Improved parking location pin prediction, now shown on a map with a P icon.
- Ehhanced response to emergency vehicles, school buses, right-of-way violators, and other rare vehicles.
- Improved handling of small animals by focusing RL training on harder examples and adding rewards for better proactive safety.
- Improved traffic light handling at complex intersections with compound lights, curved roads, and yellow light stopping.
- Includes other minor fixes and performance improvements.



Tesla FSD v14.3.8 release notes read like a project list for this course.

What the screenshot reveals

- RL stage upgraded: training pipeline changes show up as driving behavior
- Vision encoder upgraded: rare / low-visibility scenes, 3D geometry, traffic signs
- Compiler + runtime rewritten with MLIR: 20% faster reaction time + iteration speed
- Long-tail safety: emergency vehicles, school buses, animals, complex traffic lights

Why this matters

- Kernels, fusion, quantization, and memory movement determine real-time latency
- Compilers and runtimes convert models into deployable products
- Efficient data/training attacks the rare-event tail, not just average accuracy
- Final metrics are system-level: safety, reaction time, energy, and iteration velocity

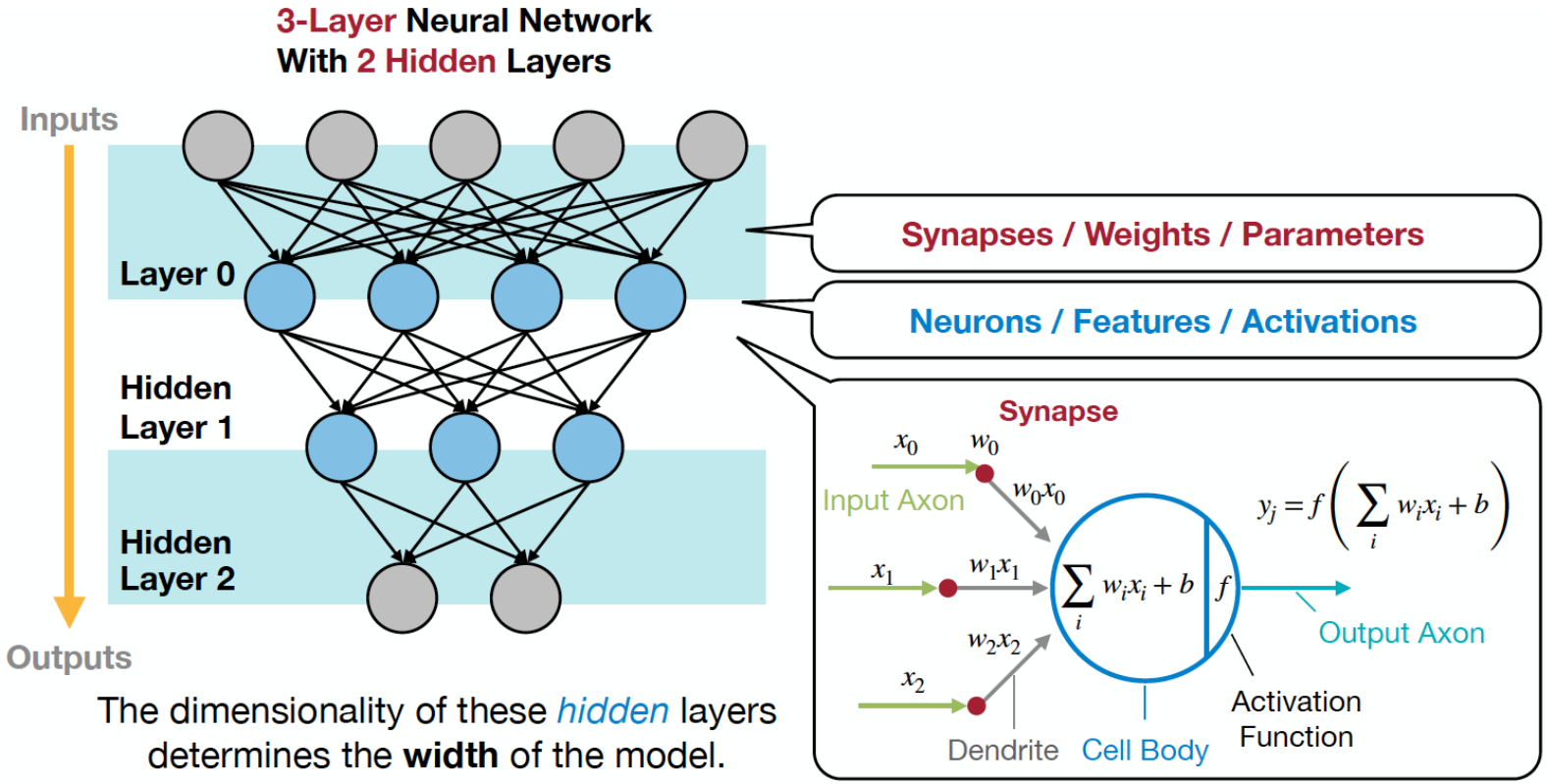
Project directions

- Profile and optimize a Transformer / vision block
- Write a Triton or MLIR fusion pass
- Quantize / prune an encoder under latency constraints
- Hard-example mining for rare events
- Tail-latency scheduler for real-time inference
- Benchmark operator choices on edge GPU / NPU

Takeaway: efficient AI is not an academic speedup; it changes reaction time, safety, and product iteration.

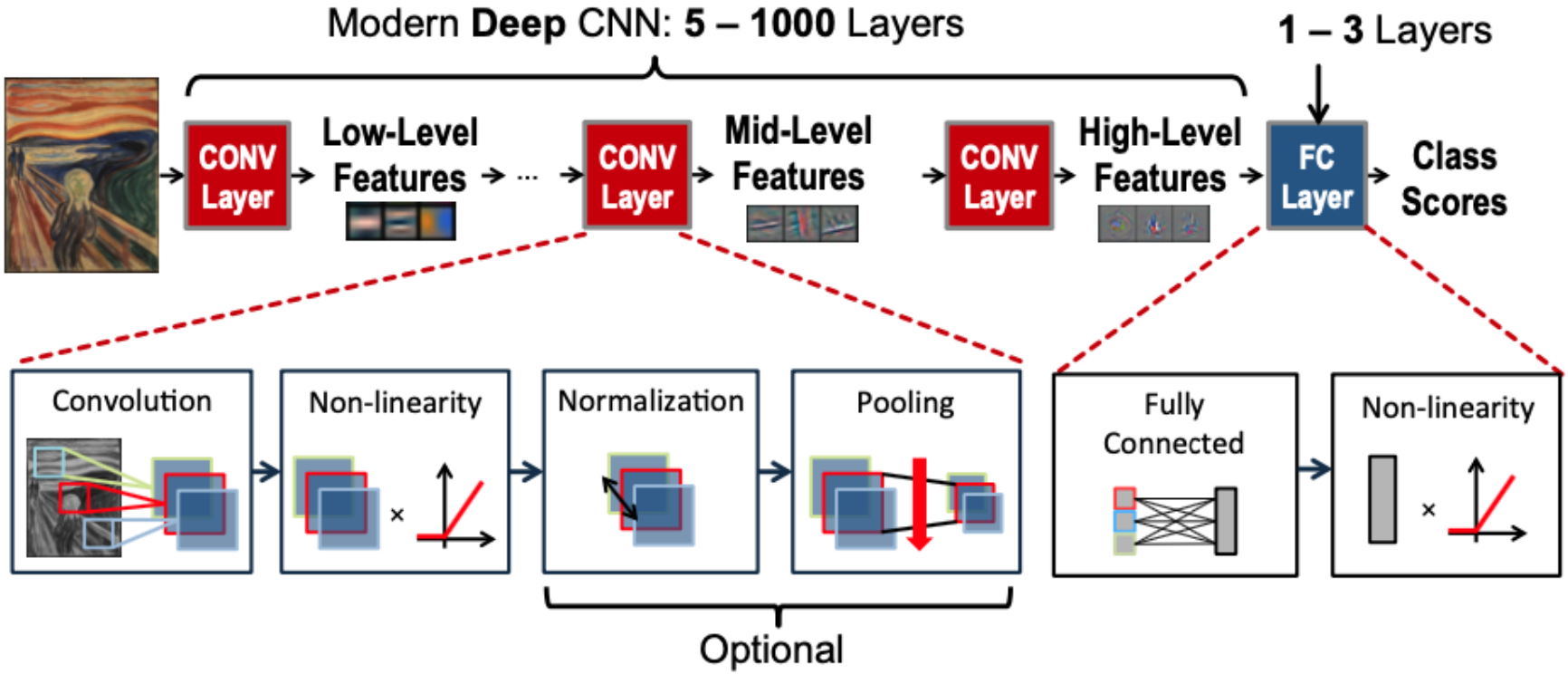
Deep Neural Networks

- **A subset of machine learning models**
  - Deep -> multiple layers
  - Linear model + nonlinear transformation



Source: Han

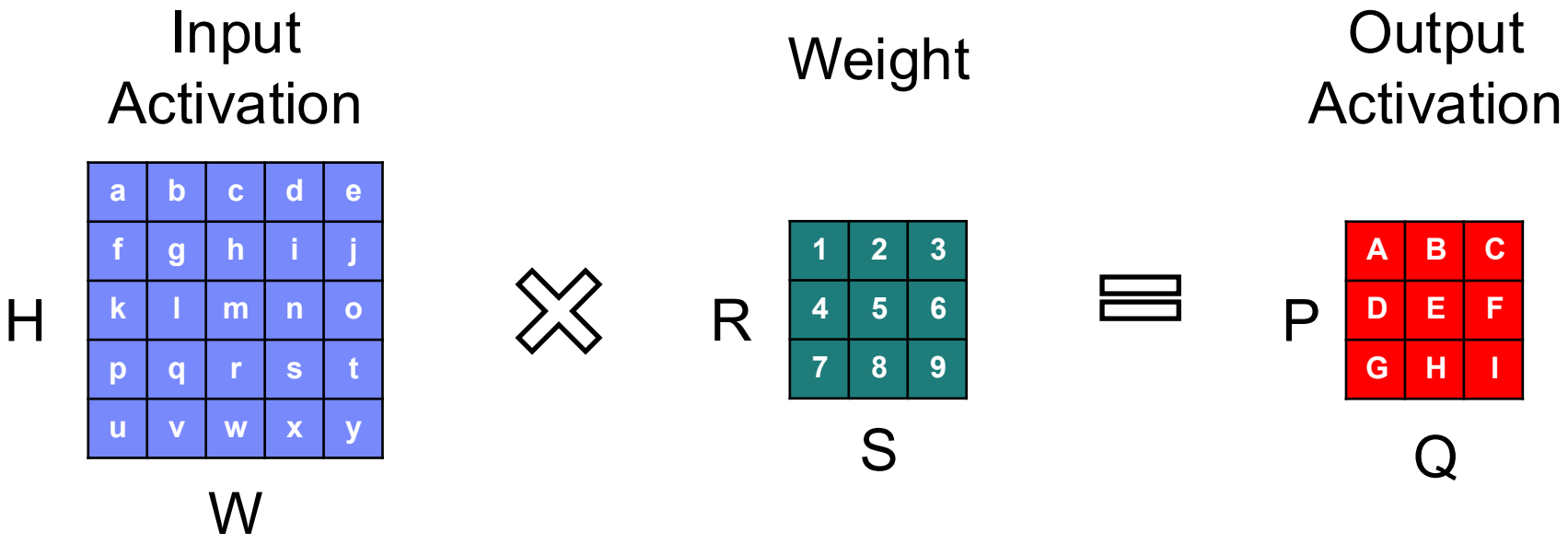
Deep Neural Networks



Eyeriss tutorial

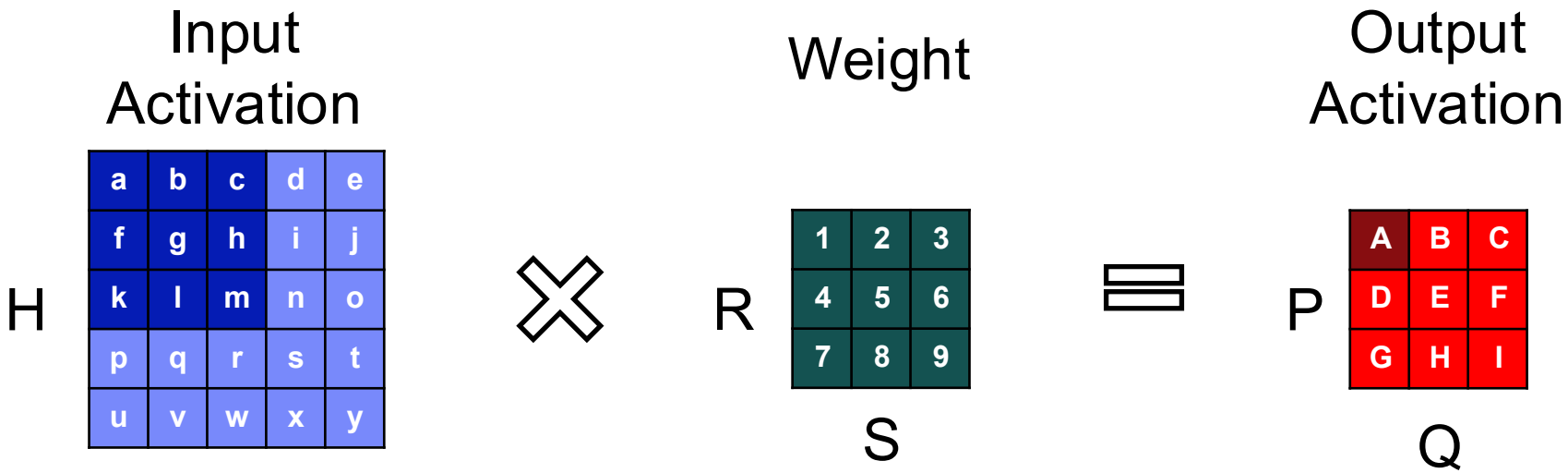
2-D Convolution

H: Height of Input Activation
W: Width of Input Activation
R: Height of Weight
S: Width of Weight
P: Height of Output Activation
Q: Width of Output Activation



2-D Convolution

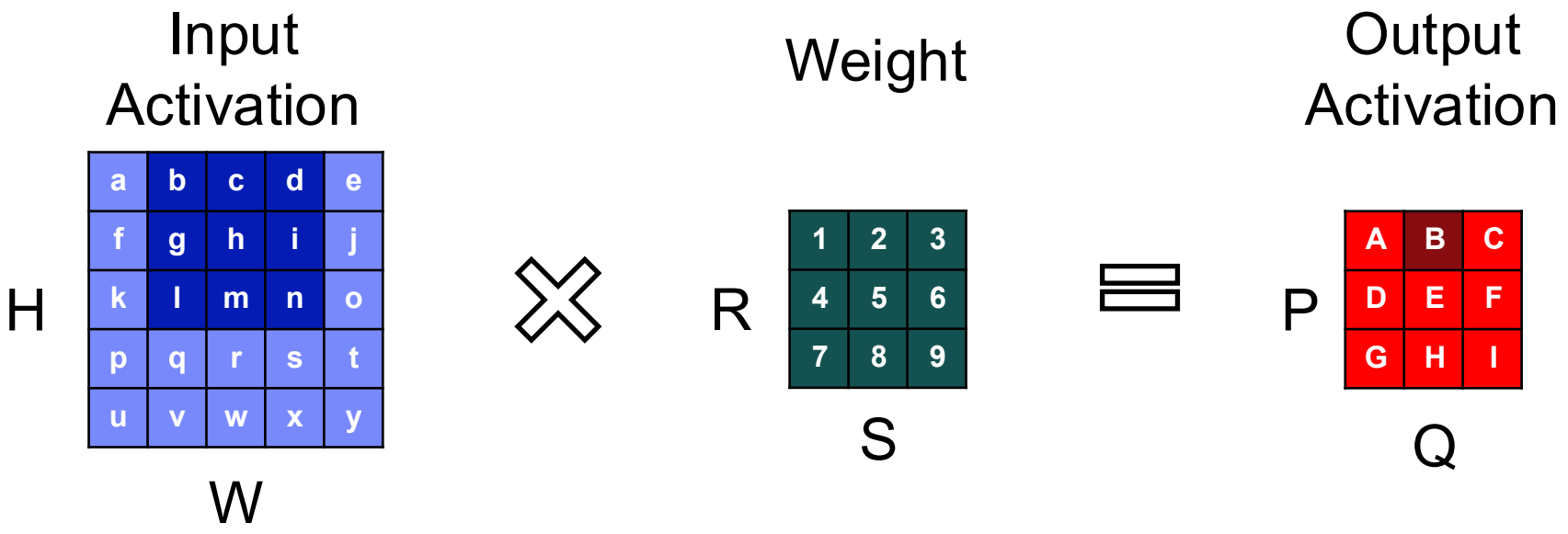
H: Height of Input Activation
W: Width of Input Activation
R: Height of Weight
S: Width of Weight
P: Height of Output Activation
Q: Width of Output Activation



$$A = a * 1 + b * 2 + c * 3$$
$$+ f * 4 + g * 5 + h * 6$$
$$+ k * 7 + l * 8 + m * 9$$

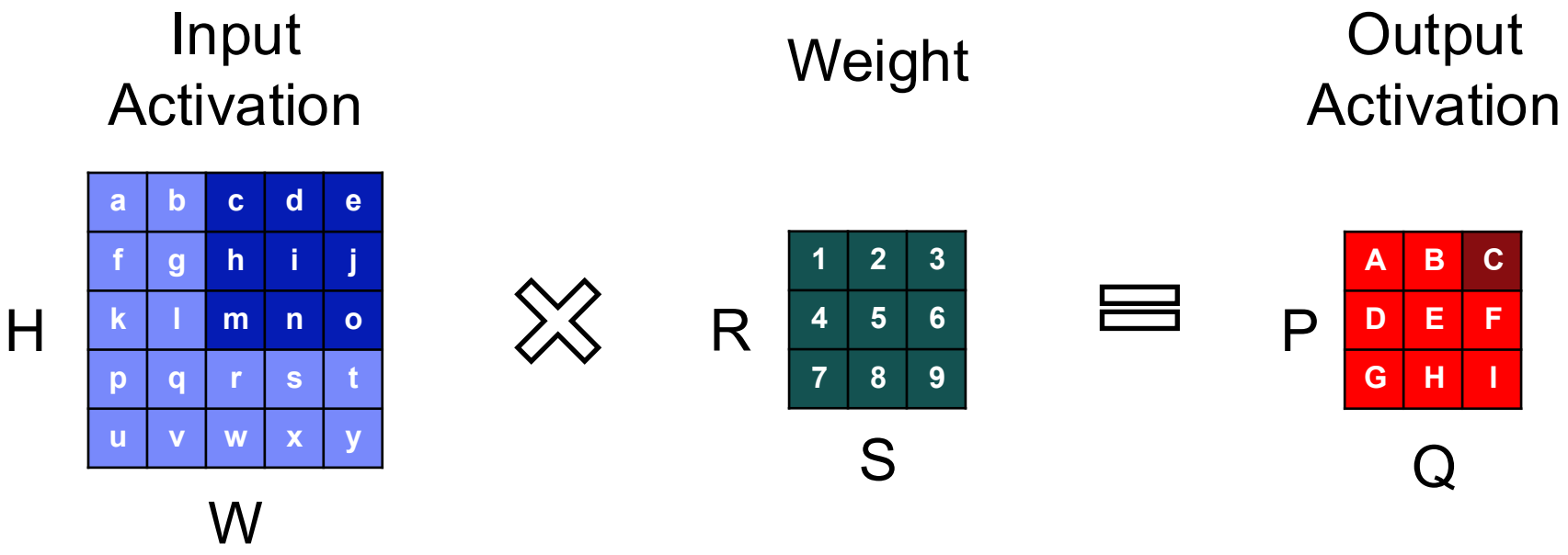
2-D Convolution (stride = 1)

H: Height of Input Activation
W: Width of Input Activation
R: Height of Weight
S: Width of Weight
P: Height of Output Activation
Q: Width of Output Activation
stride: # of rows/columns traversed per step



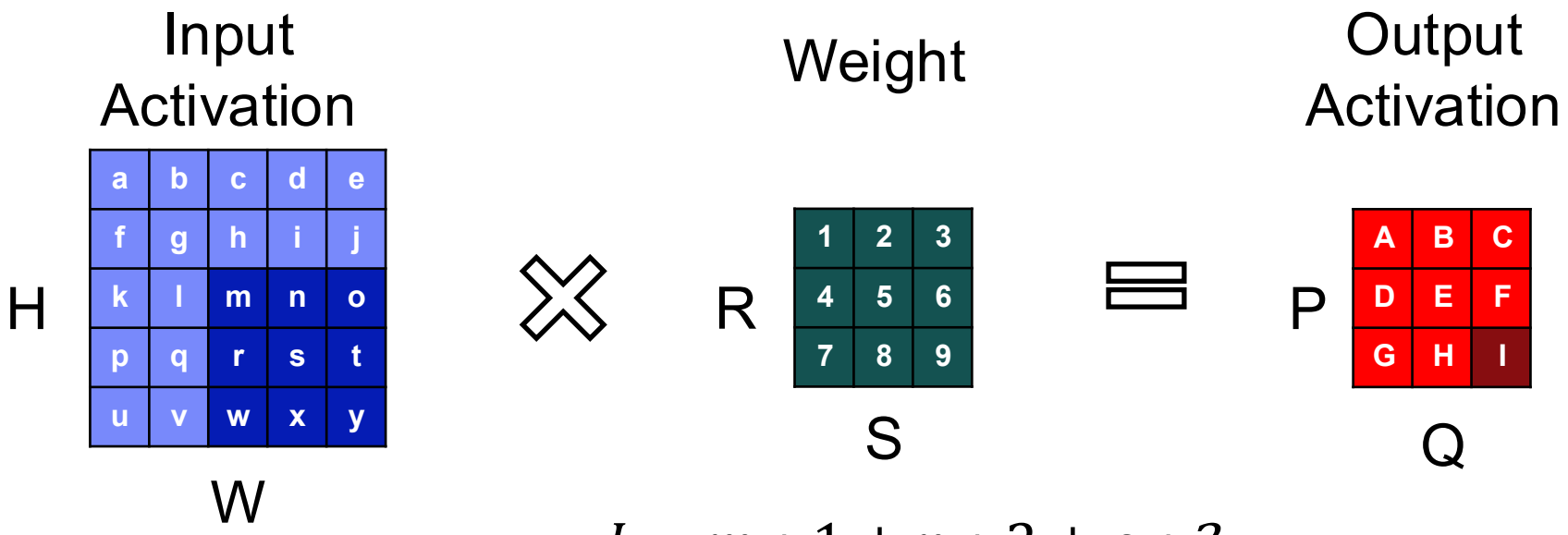
2-D Convolution (stride = 1)

H: Height of Input Activation
W: Width of Input Activation
R: Height of Weight
S: Width of Weight
P: Height of Output Activation
Q: Width of Output Activation
stride: # of rows/columns traversed per step



2-D Convolution (stride = 1)

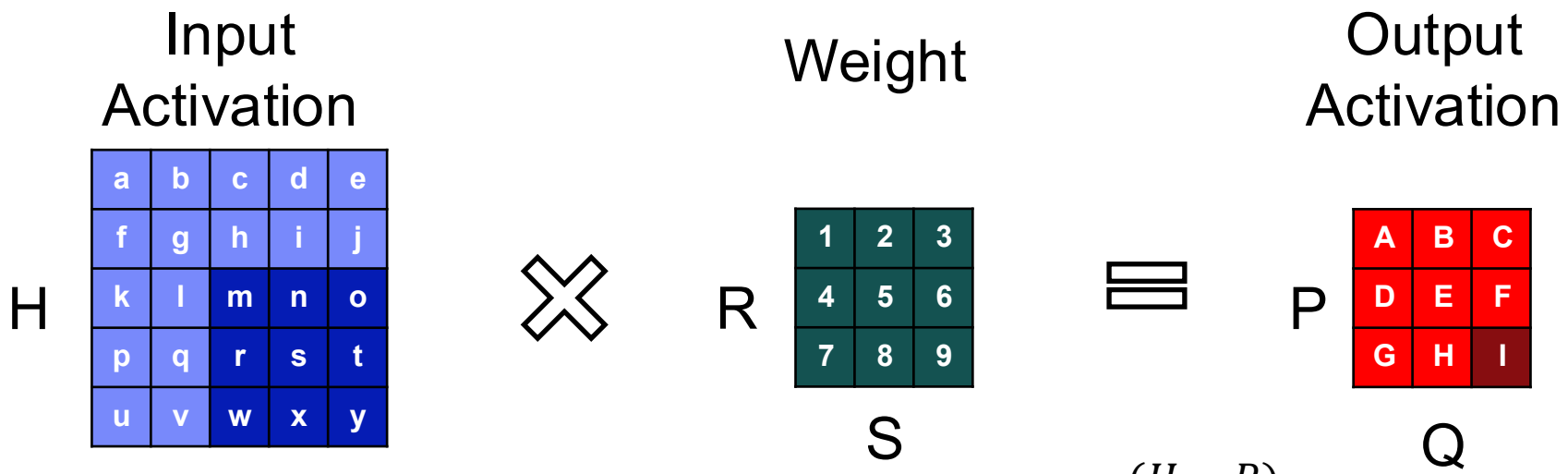
H: Height of Input Activation
W: Width of Input Activation
R: Height of Weight
S: Width of Weight
P: Height of Output Activation
Q: Width of Output Activation
stride: # of rows/columns traversed per step



$$I = m * 1 + n * 2 + o * 3 + r * 4 + s * 5 + t * 6 + w * 7 + x * 8 + y * 9$$

2-D Convolution (stride = 1)

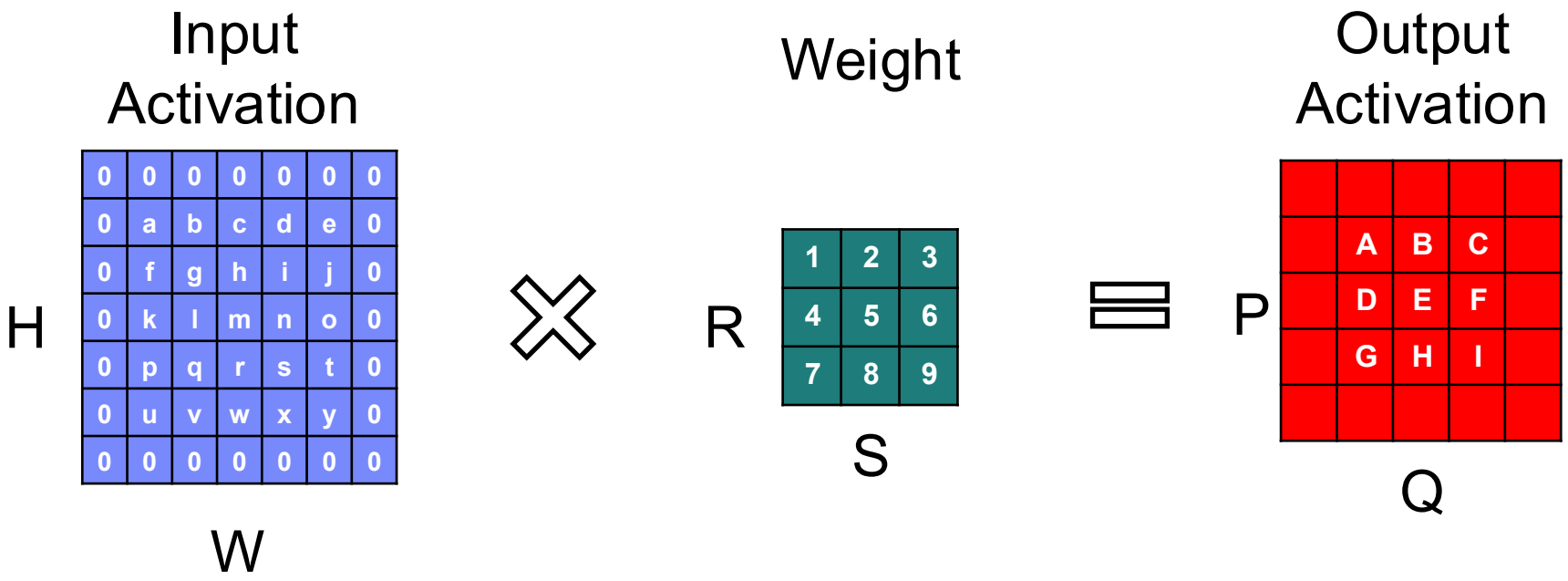
H: Height of Input Activation
W: Width of Input Activation
R: Height of Weight
S: Width of Weight
P: Height of Output Activation
Q: Width of Output Activation
stride: # of rows/columns traversed per step



$$P = \frac{(H - R)}{stride} + 1$$

$$Q = \frac{(W - S)}{stride} + 1$$

2-D Convolution (stride = 1, padding = 1)

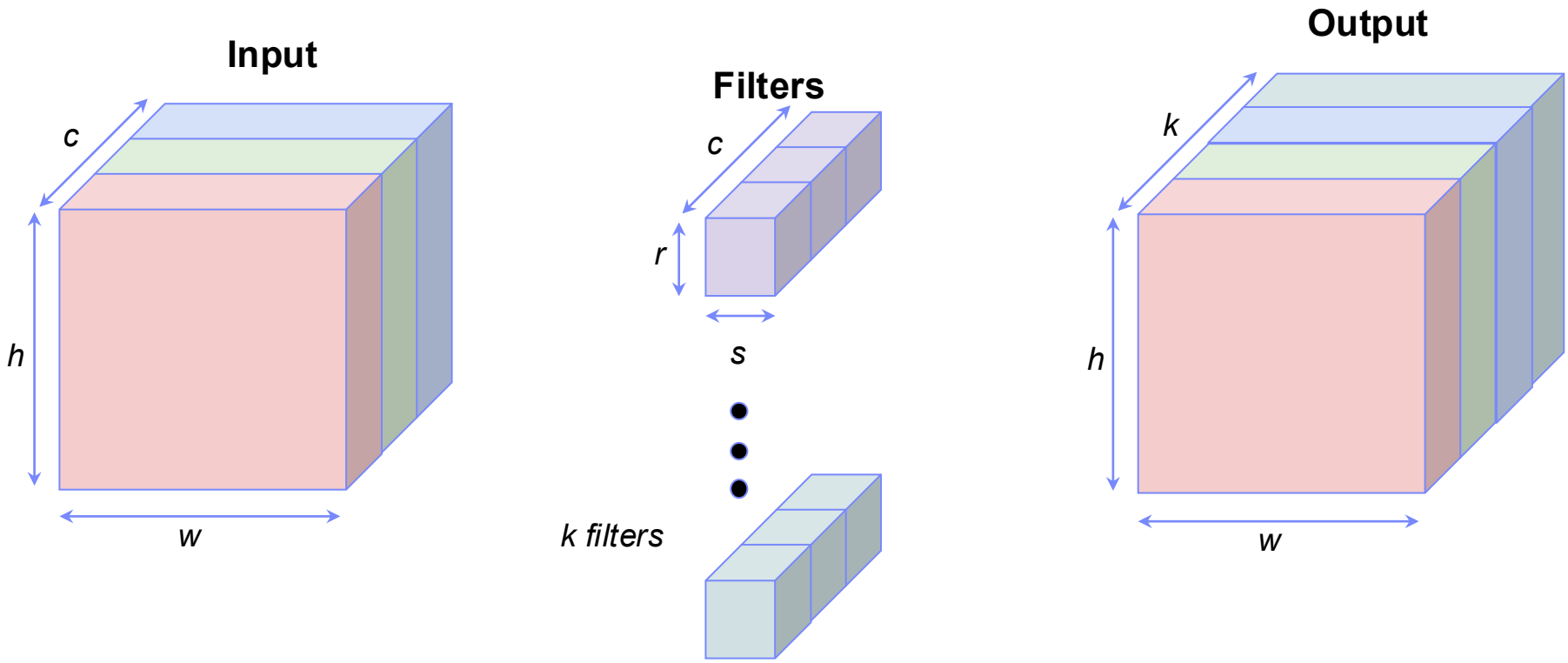


H: Height of Input Activation
W: Width of Input Activation
R: Height of Weight
S: Width of Weight
P: Height of Output Activation
Q: Width of Output Activation
stride: # of rows/columns traversed per step
padding: # of zero rows/columns added

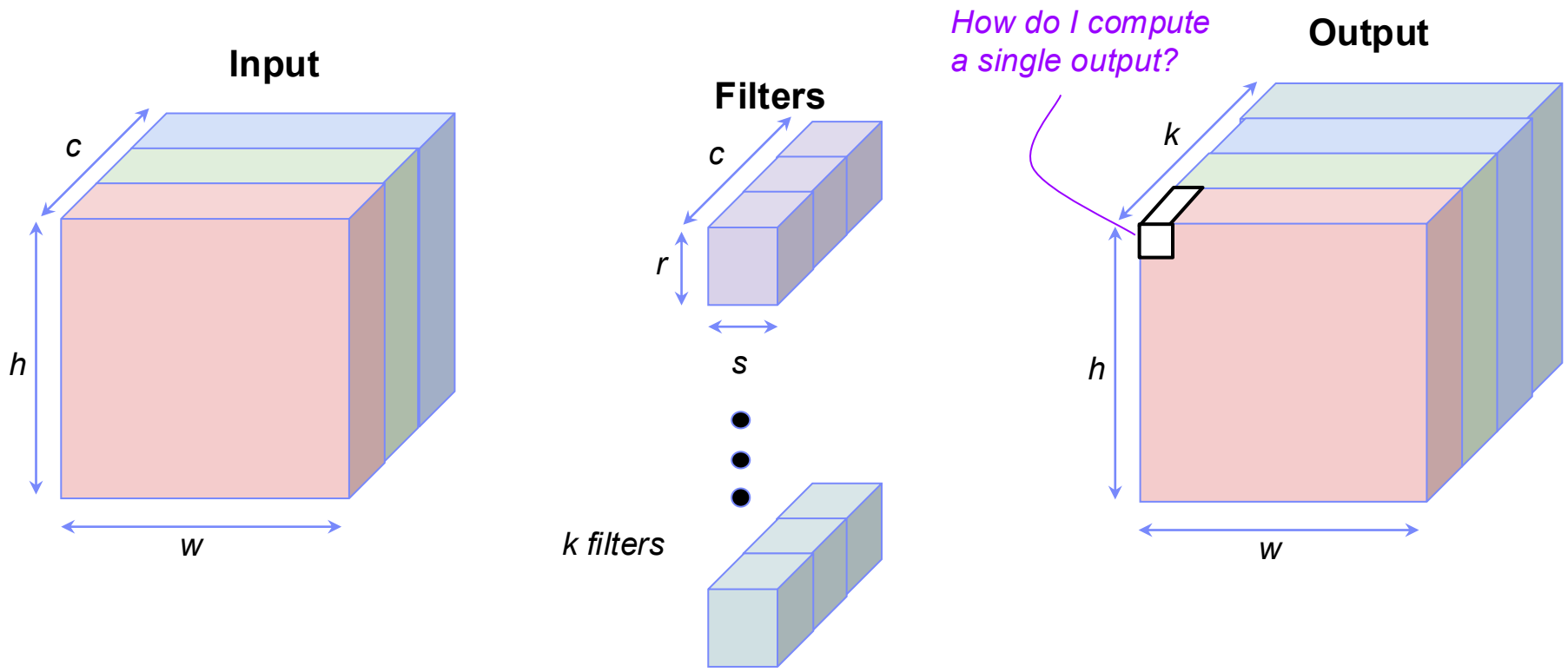
$$P = \frac{(H - R + 2 * pad)}{stride} + 1$$

$$Q = \frac{(W - S + 2 * pad)}{stride} + 1$$

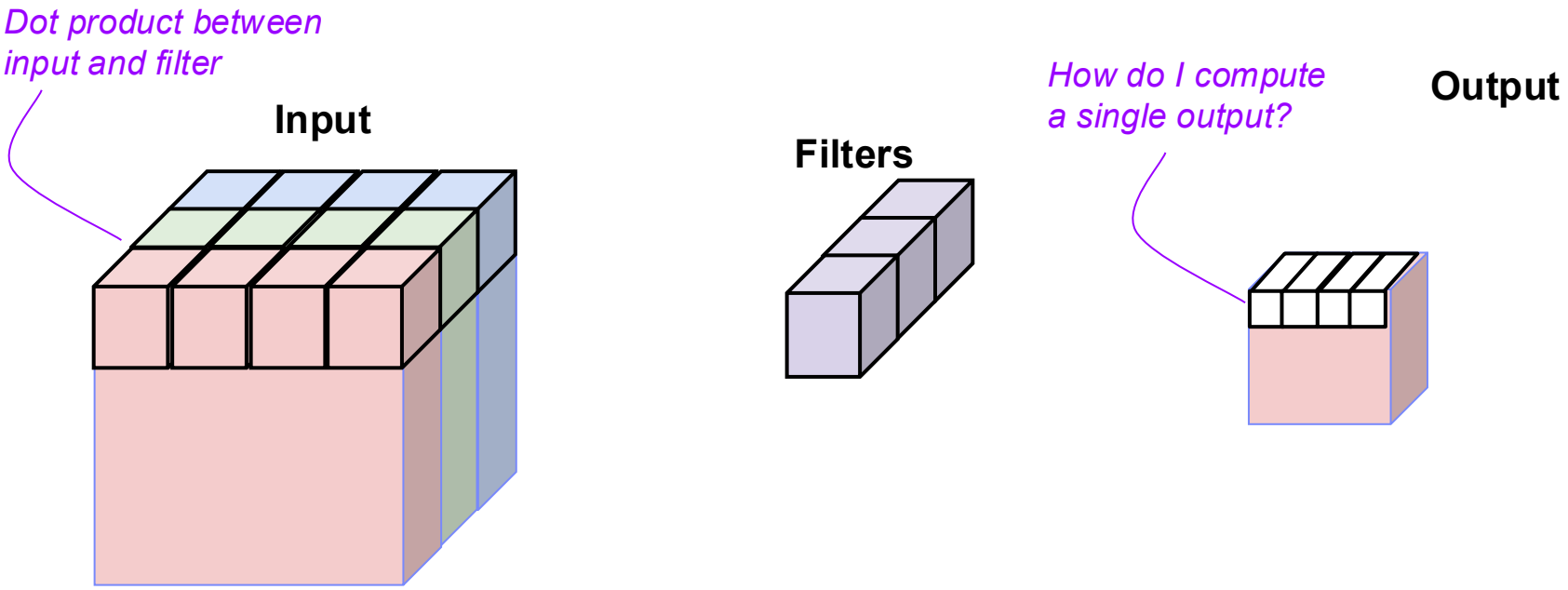
3D Convolution



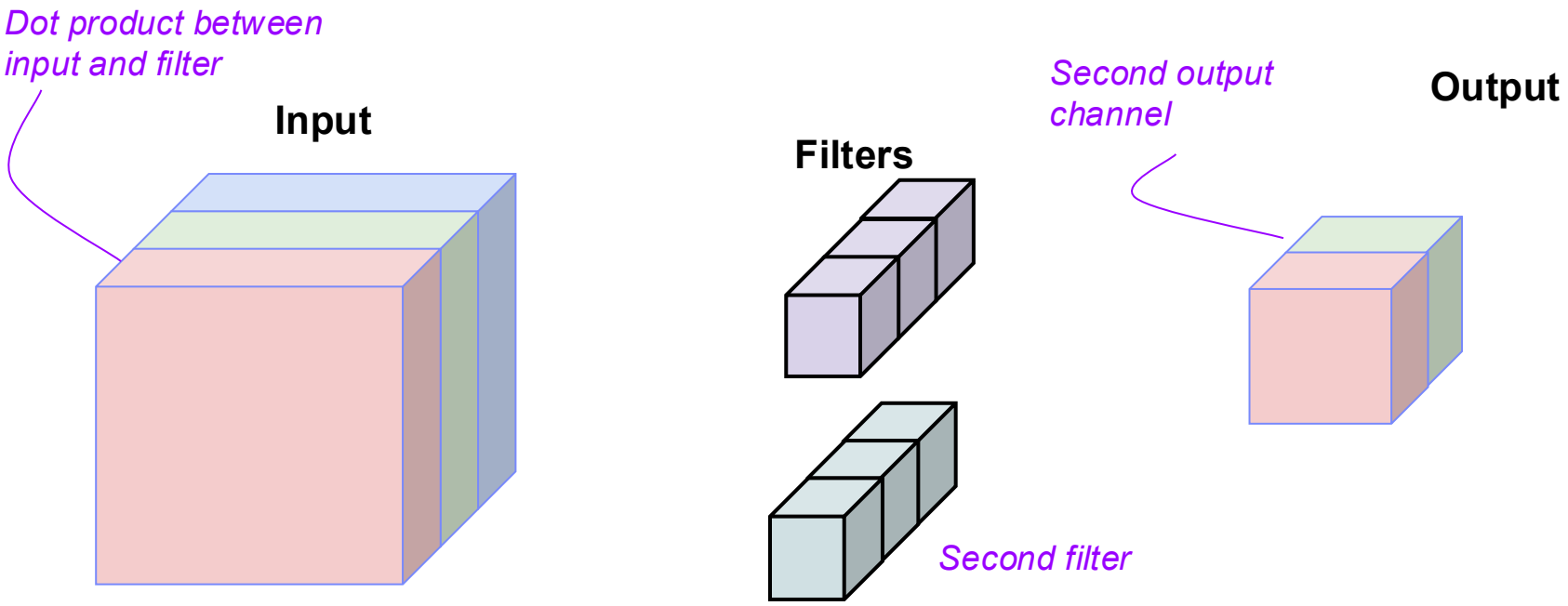
3D Convolution



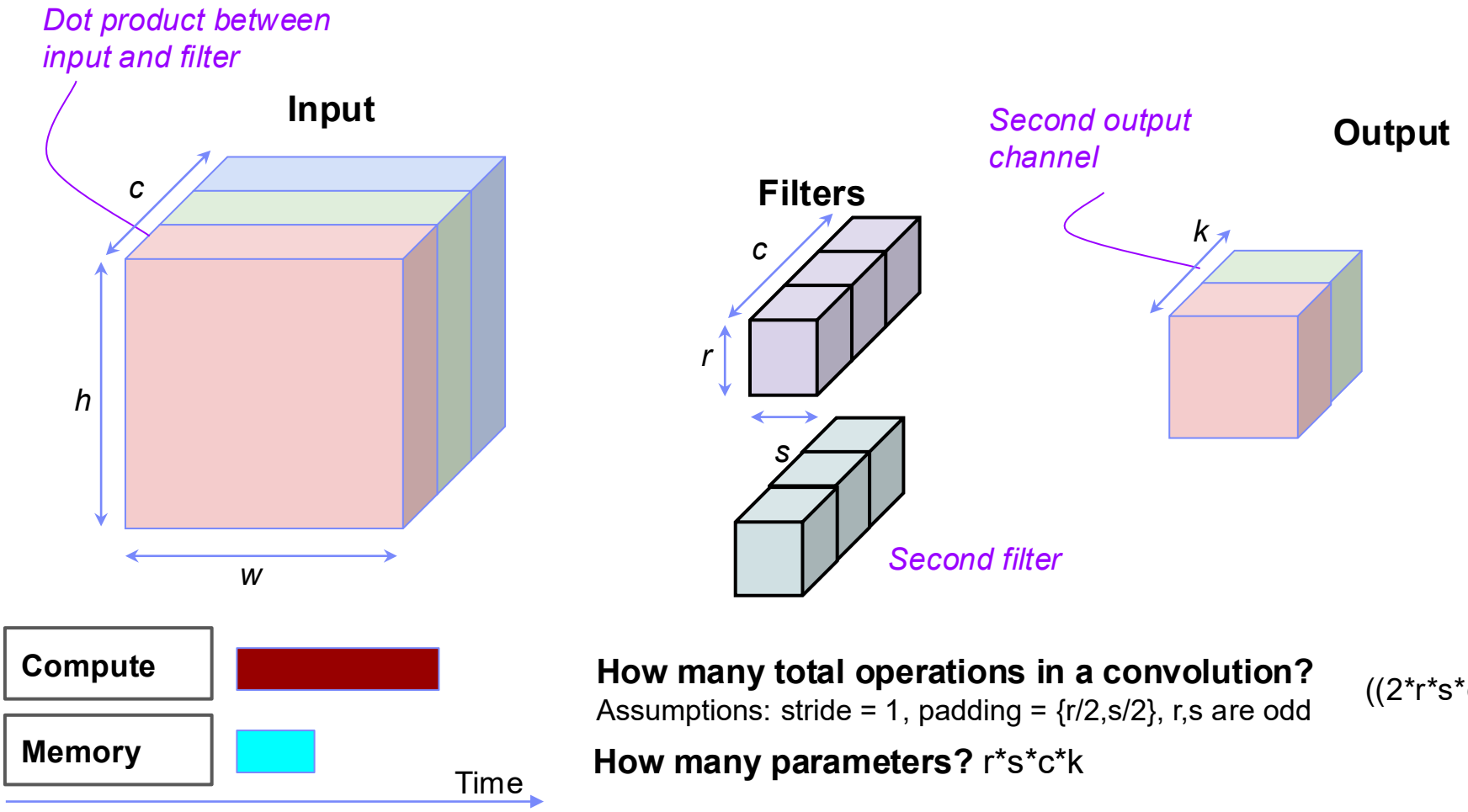
3D Convolution



3D Convolution



3D Convolution

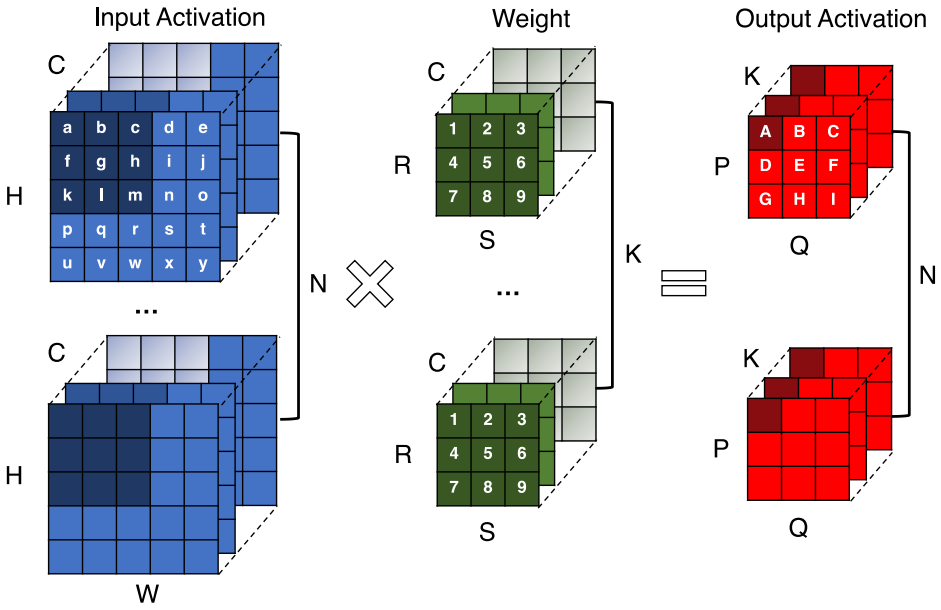


How many total operations in a convolution?
Assumptions: stride = 1, padding = {r/2,s/2}, r,s are odd

((2*r*s*c)*h*w)*k

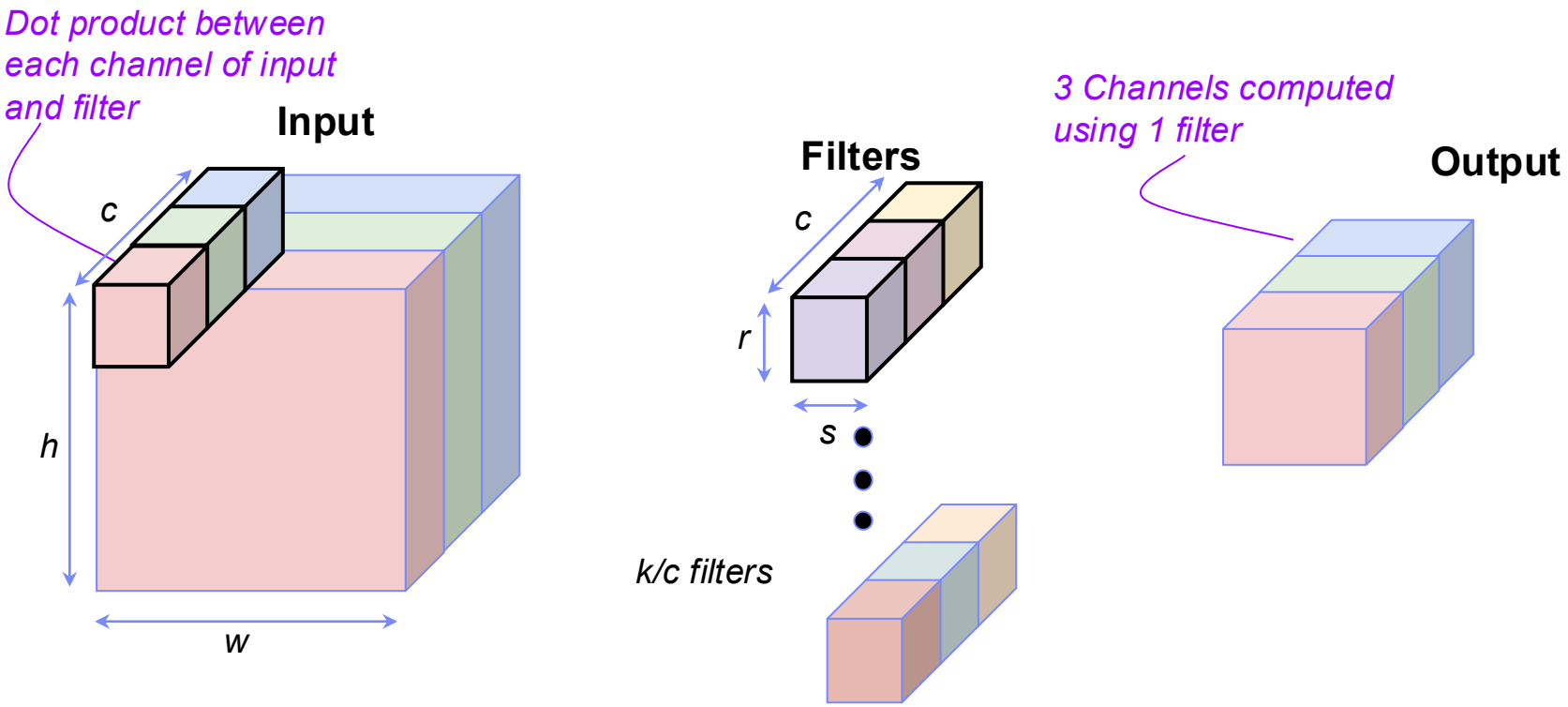
How many parameters? r*s*c*k

Convolution Loop Nest



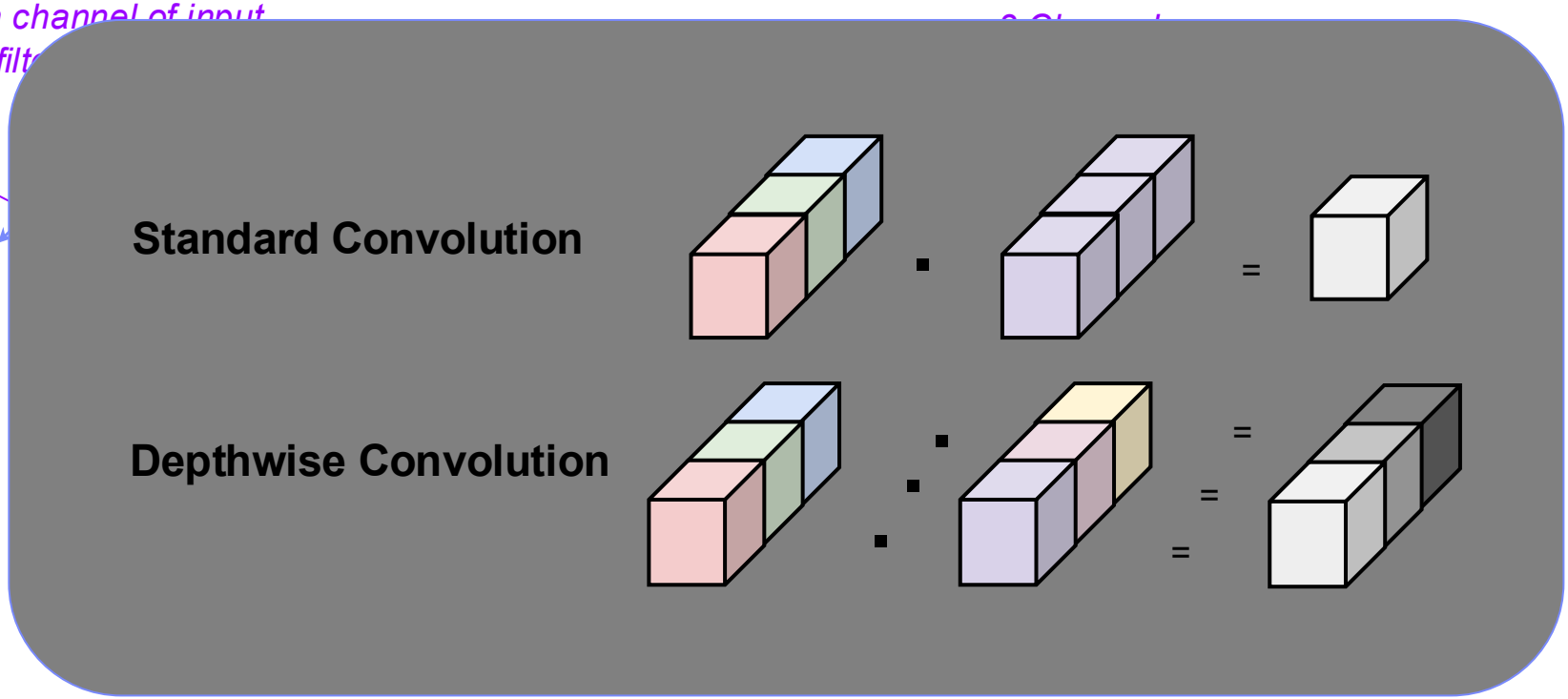

```
for (n=0; n<N; n++) {
  for (k=0; k<K; k++) {
    for (p=0; p<P; p++) {
      for (q=0; q<Q; q++) {
        OA[n][k][p][q]= 0;
        for (r=0; r<R; r++) {
          for (s=0; s<S; s++) {
            for (c=0; c<C; c++) {
              h = p * stride - pad + r;
              w = q * stride - pad + s;
              OA[n][k][p][q] +=
                                    IA[n][c][h][w]
                              * W[k][c][r][s];
            }
          }
        }
        OA[n][k][p][q]= Activation(OA[n][k][p][q]);
      }
    }
  }
}
```

Depthwise Convolution



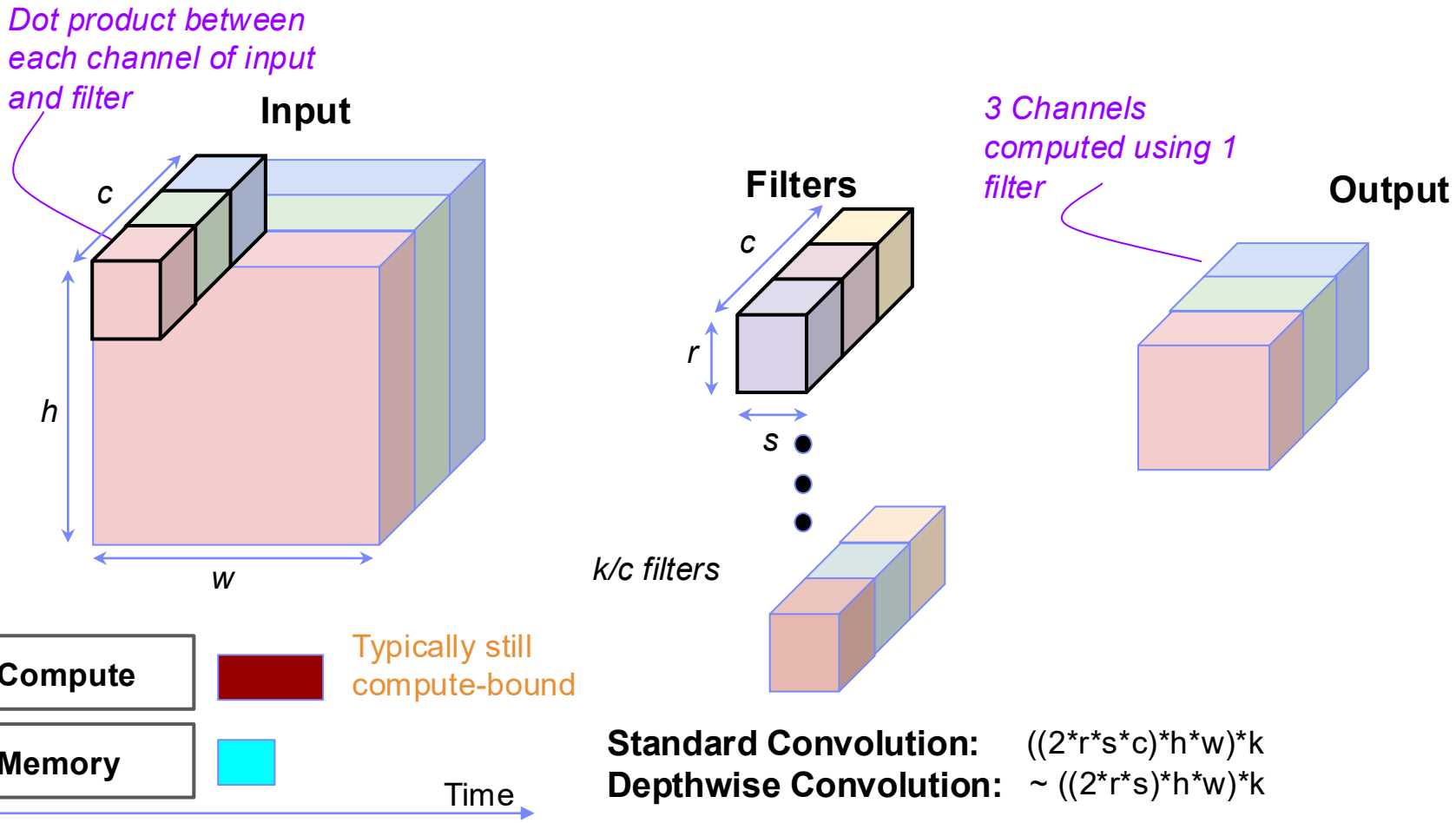
Depthwise Convolution

Dot product between
each channel of input
and filt



Standard Convolution: ((2*r*s*c)*h*w)*k
Depthwise Convolution: ~ ((2*r*s)*h*w)*k (k=c)

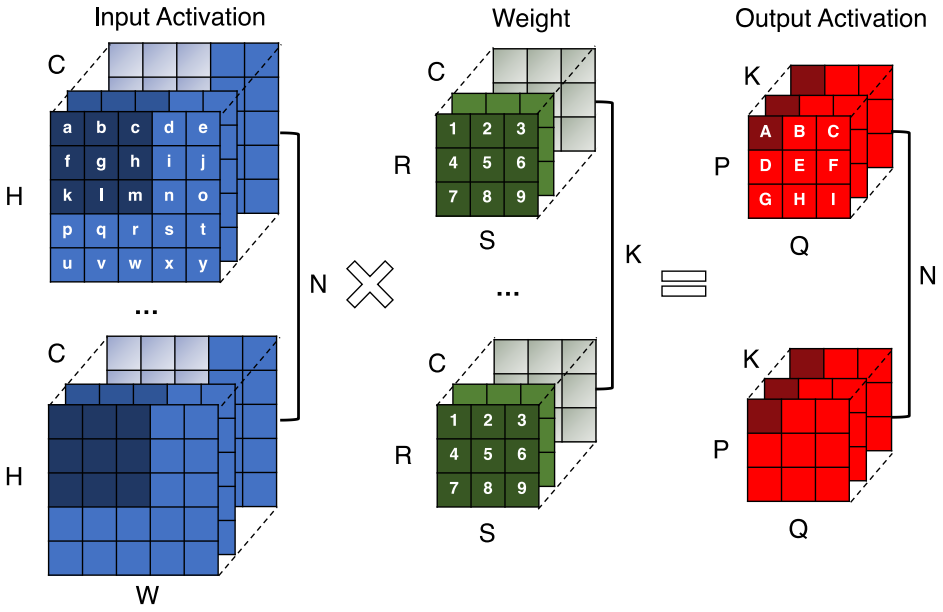
Depthwise Convolution



Standard Convolution: ((2*r*s*c)*h*w)*k

Depthwise Convolution: ~ ((2*r*s)*h*w)*k

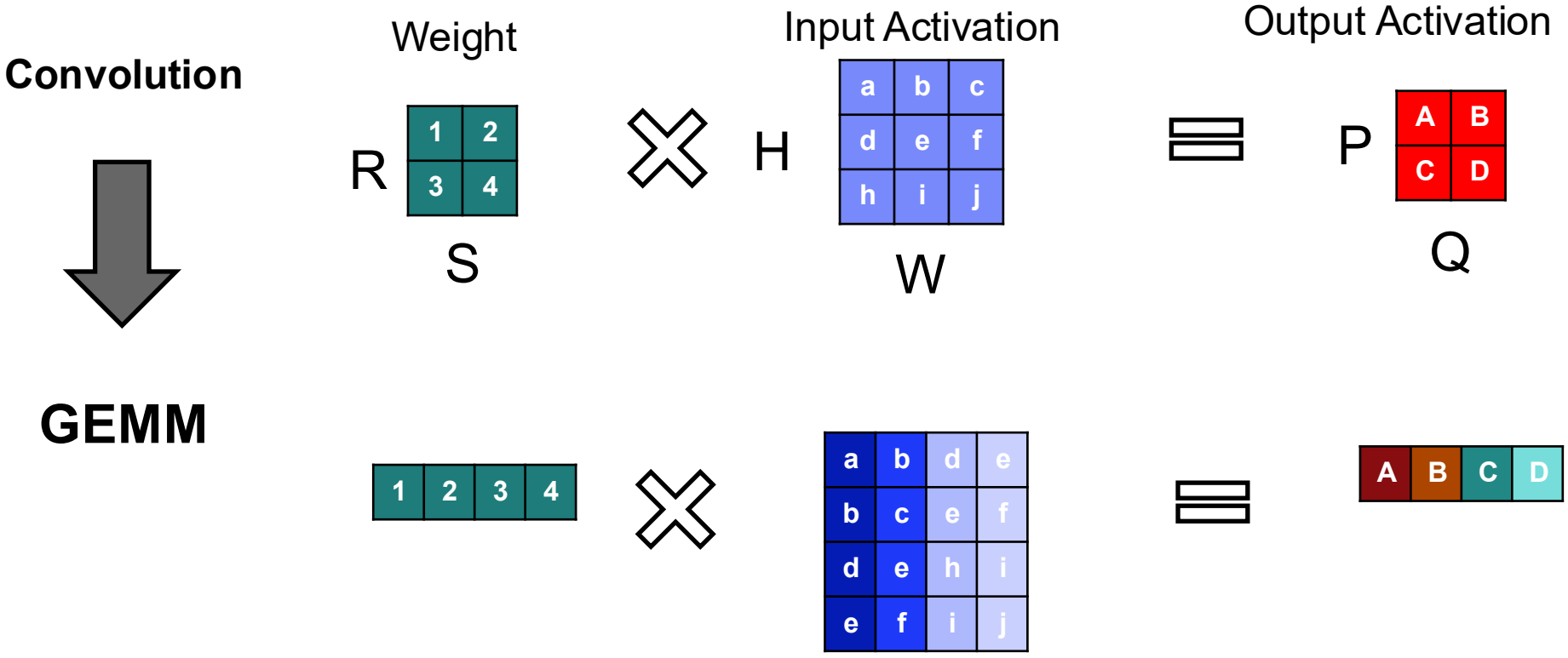
Convolution Transformation
Option 1: Direct Convolution




```
for (n=0; n<N; n++) {
  for (k=0; k<K; k++) {
    for (p=0; p<P; p++) {
      for (q=0; q<Q; q++) {
        OA[n][k][p][q]= 0;
        for (r=0; r<R; r++) {
          for (s=0; s<S; s++) {
            for (c=0; c<C; c++) {
              h = p * stride - pad + r;
              w = q * stride - pad + s;
              OA[n][k][p][q] +=
                                    IA[n][c][h][w]
                              * W[k][c][r][s];
            }
          }
        }
          OA[n][k][p][q]= Activation(OA[n][k][p][q]);
      }
    }
  }
}
```

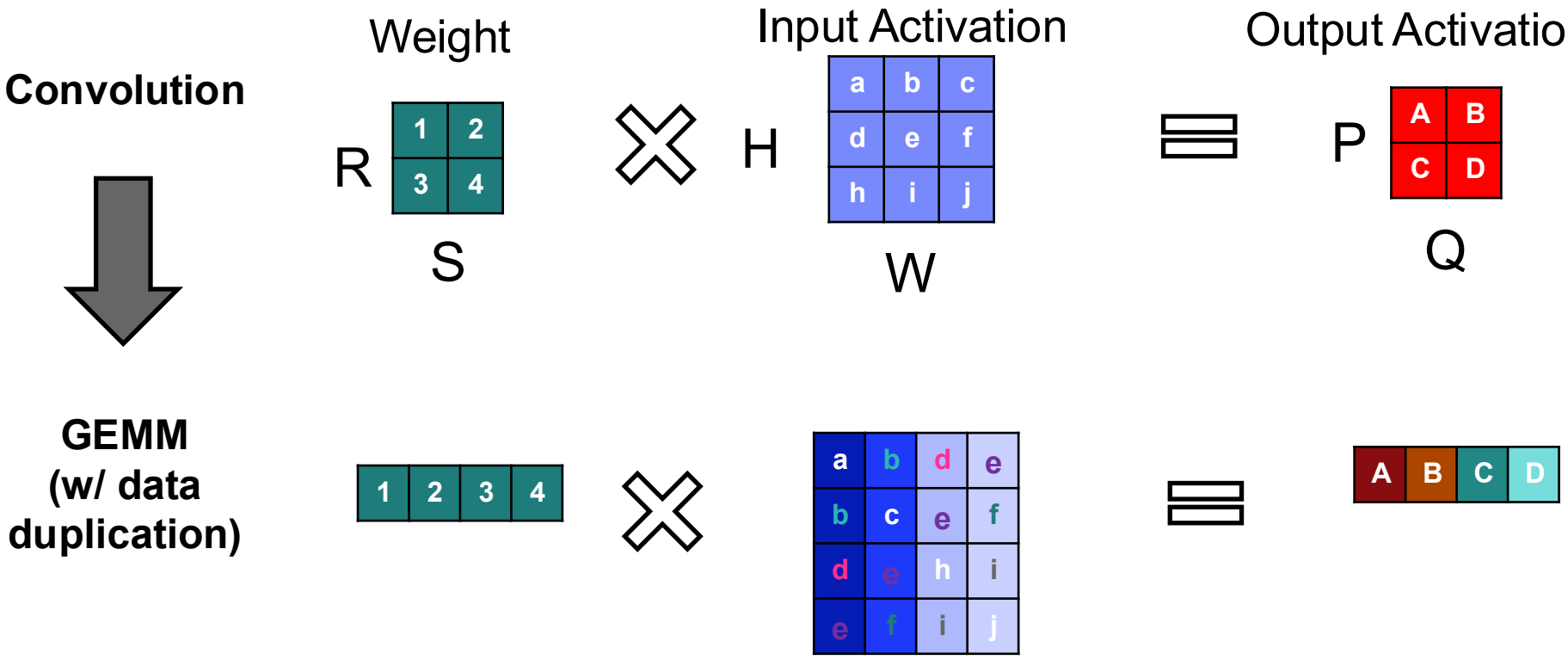
Convolution Transformation Option 2: GEMM

- **Converting convolution to GEMM via im2col**



Convolution Transformation Option 2: GEMM

- **Converting convolution to GEMM via im2col**



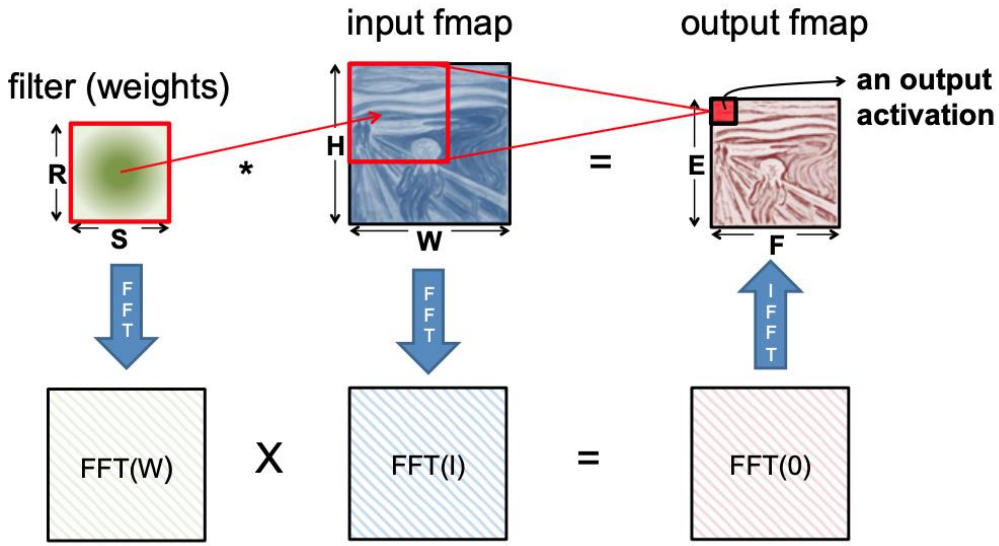
Convolution Transformation
Option 3: FFT-based Convolution

- **Convolution theorem:**
  Convolution in the time domain is equivalent to point-wise multiply in the frequency domain.

$$f * g = \mathcal{F}^{-1}\{\mathcal{F}\{f\} \cdot \mathcal{F}\{g\}\}$$

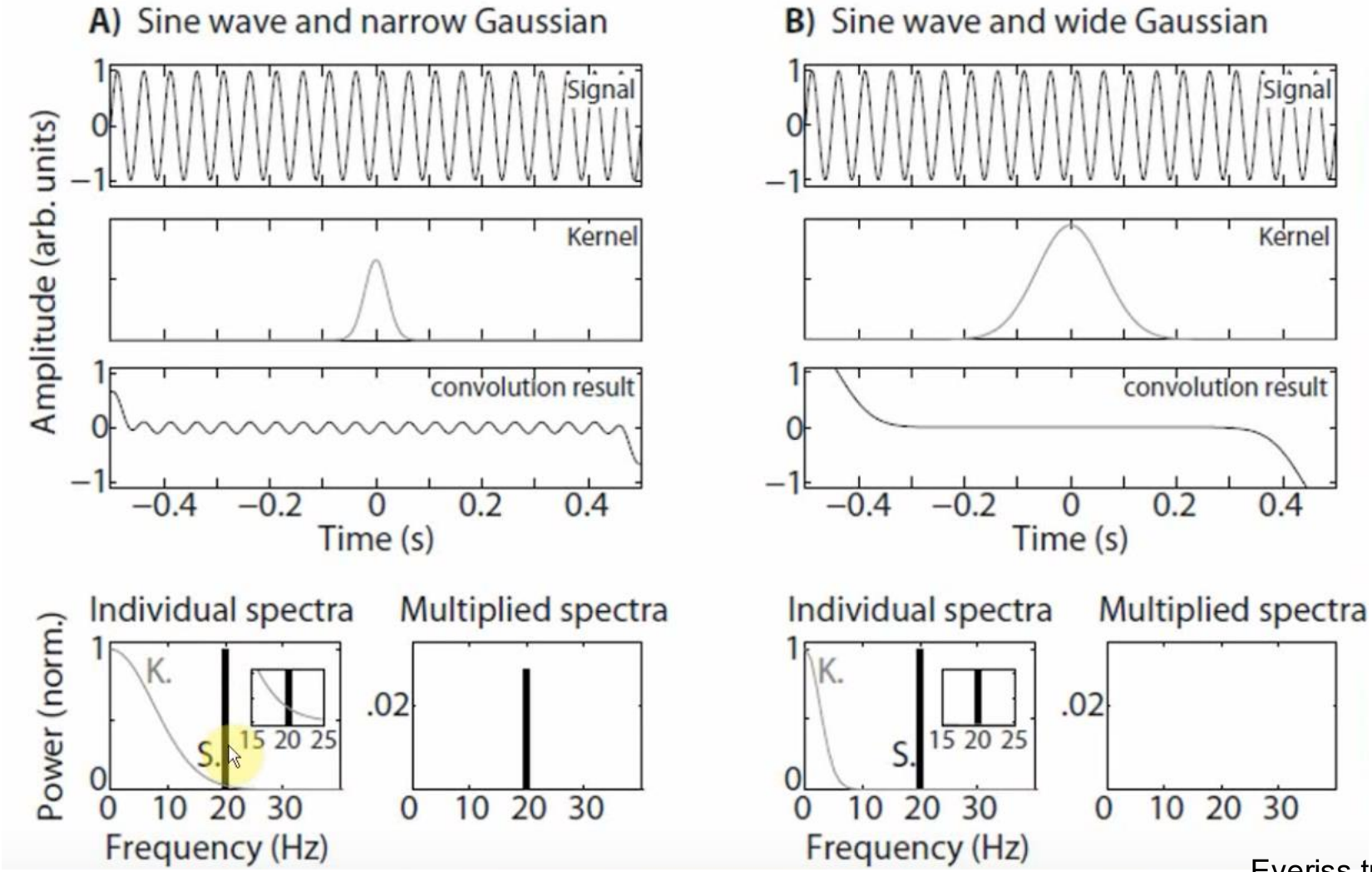
$\mathcal{F}\{f\}$ and $\mathcal{F}\{g\}$ are the Fourier transforms of f and g

The asterisk denotes convolution, not multiplication.



Eyeriss tutorial

Convolution Transformation
Option 3: FFT-based Convolution



Eyeriss tutorial

Convolution Transformation
Option 3: FFT-based Convolution

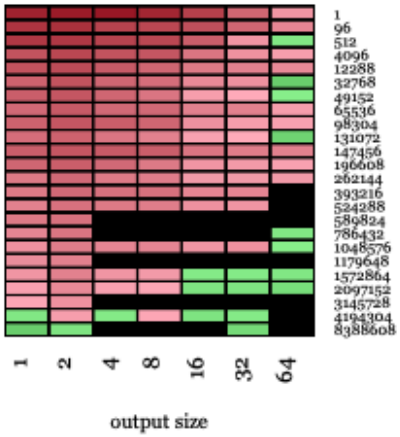


Figure 1: 3 × 3 kernel (K40m)

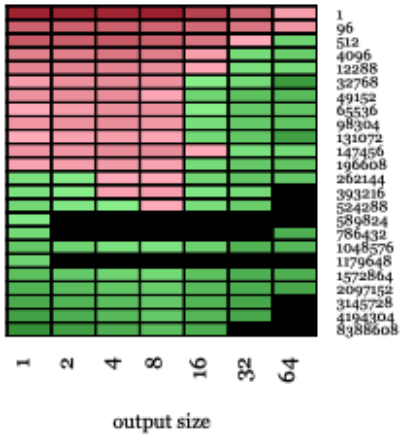


Figure 2: 5 × 5 kernel (K40m)

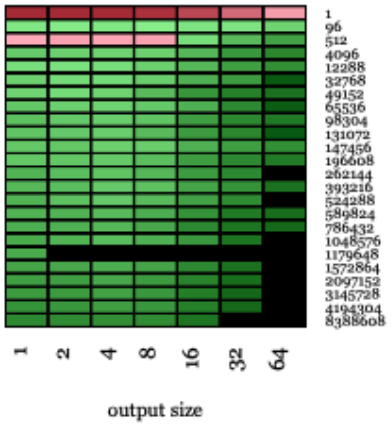


Figure 5: 11 × 11 kernel (K40m)

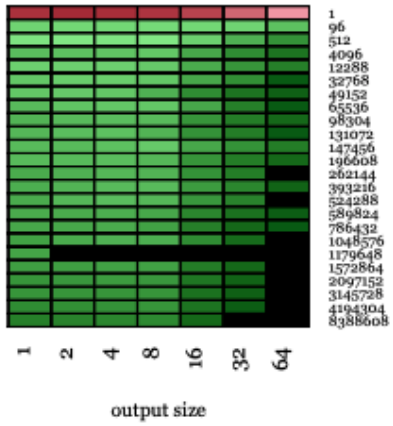


Figure 6: 13 × 13 kernel (K40m)

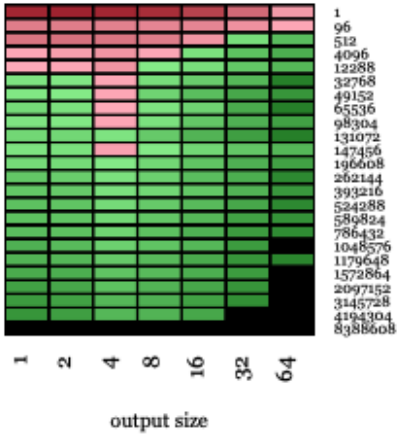


Figure 3: 7 × 7 kernel (K40m)

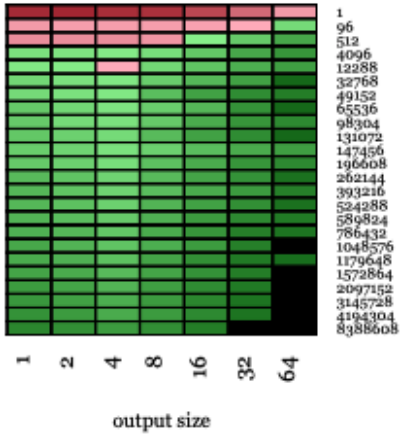
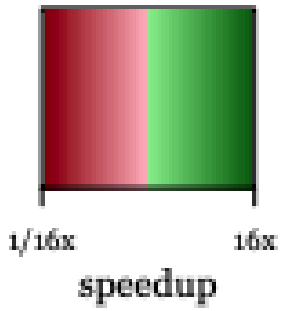


Figure 4: 9 × 9 kernel (K40m)



Fast Convolutional Nets with fbfft: A GPU Performance Evaluation

Convolution Transformation
Option 4: Winograd Transform

- **Re-association of intermediate values to reduce # of multiplications.**
- **Works well for 3x3 convolution.**

$$F(2,3) = \begin{bmatrix} d_0 & d_1 & d_2 \\ d_1 & d_2 & d_3 \end{bmatrix} \begin{bmatrix} g_0 \\ g_1 \\ g_2 \end{bmatrix} = \begin{bmatrix} m_1 + m_2 + m_3 \\ m_2 - m_3 - m_4 \end{bmatrix} \tag{5}$$

where

$$m_1 = (d_0 - d_2)g_0 \qquad m_2 = (d_1 + d_2)\frac{g_0 + g_1 + g_2}{2}$$
$$m_4 = (d_1 - d_3)g_2 \qquad m_3 = (d_2 - d_1)\frac{g_0 - g_1 + g_2}{2}$$

$$Y = A^T\left[(Gg) \odot (B^T d)\right] \tag{6}$$

$$B^T = \begin{bmatrix} 1 & 0 & -1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & -1 & 1 & 0 \\ 0 & 1 & 0 & -1 \end{bmatrix}$$

$$G = \begin{bmatrix} 1 & 0 & 0 \\ \frac{1}{2} & \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} & \frac{1}{2} \\ 0 & 0 & 1 \end{bmatrix} \tag{7}$$

$$A^T = \begin{bmatrix} 1 & 1 & 1 & 0 \\ 0 & 1 & -1 & -1 \end{bmatrix}$$

$$g = \begin{bmatrix} g_0 & g_1 & g_2 \end{bmatrix}^T$$

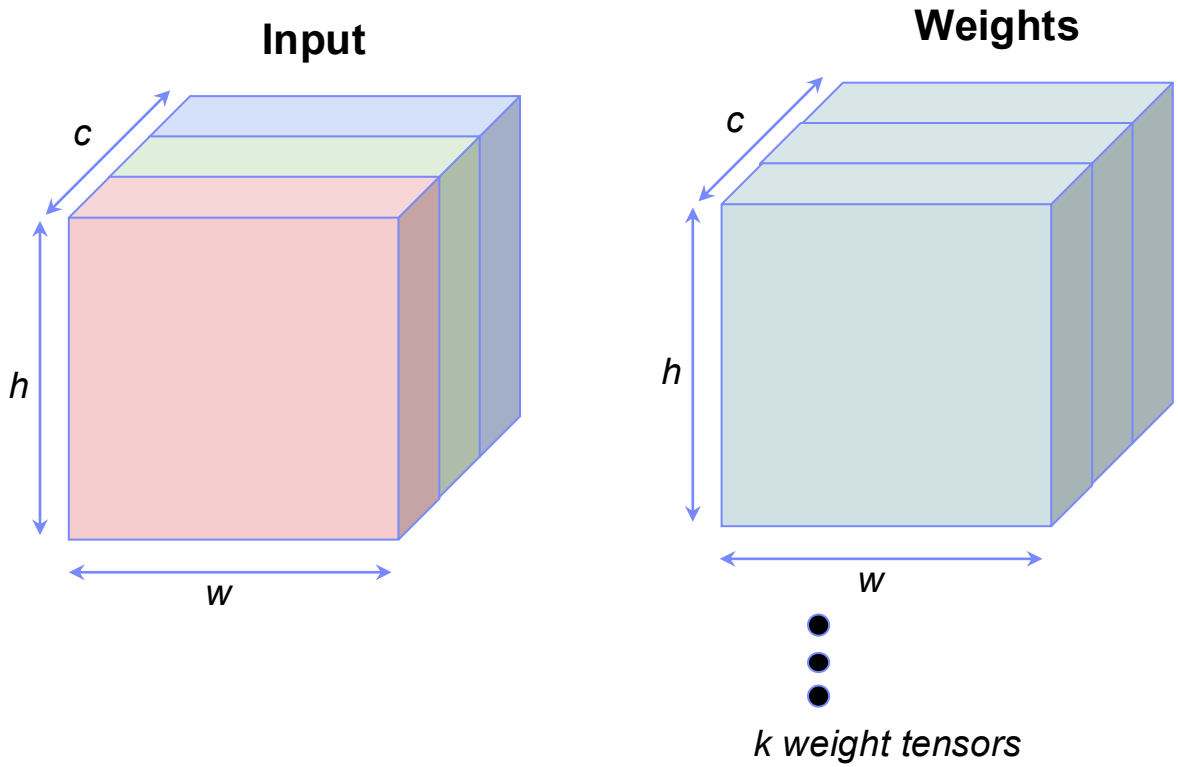
$$d = \begin{bmatrix} d_0 & d_1 & d_2 & d_3 \end{bmatrix}^T$$

Before: 6 MULs, 4 ADDs
After:

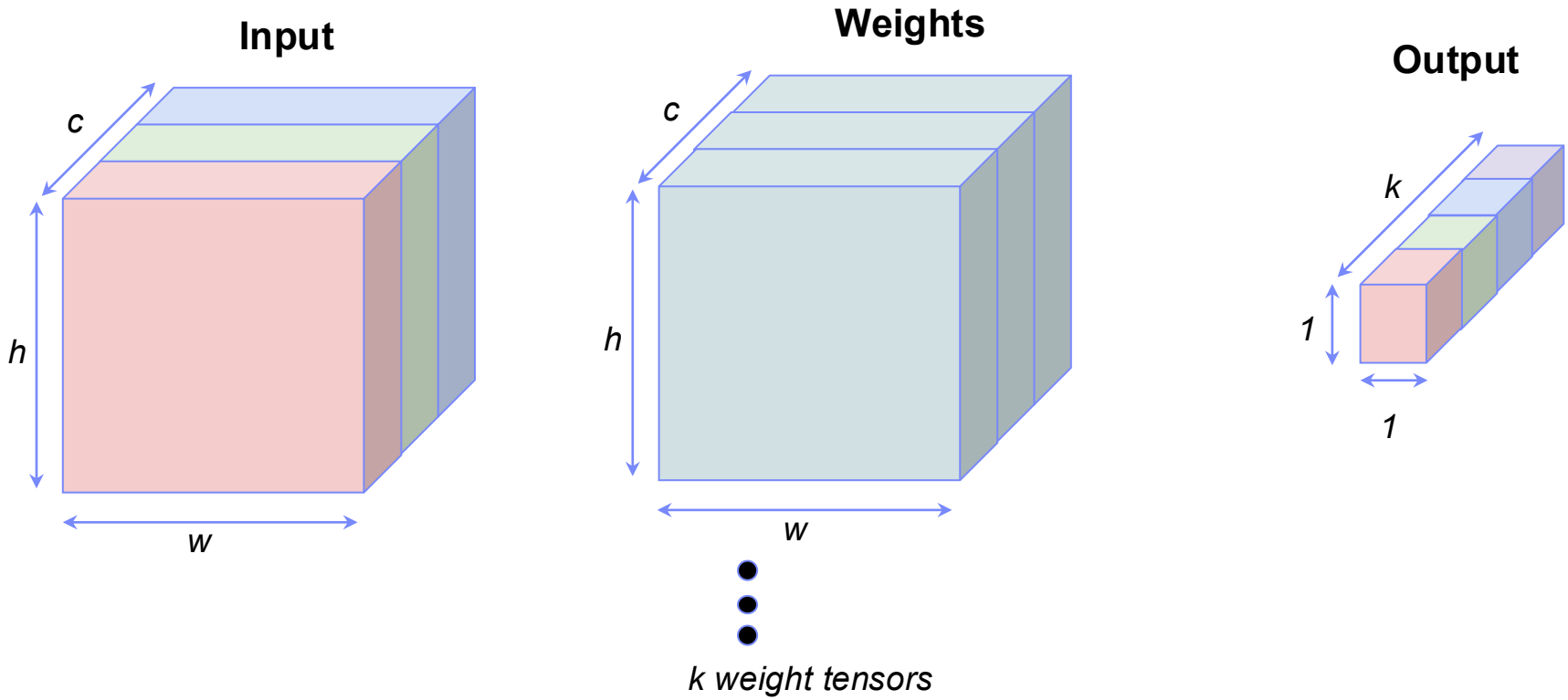
- IA (d): 4 ADDs
- W (g): 3 ADDs, 2 MULs
- OA (m): 4 MULs, 4 ADDs

Fast Algorithms for Convolutional Neural Networks

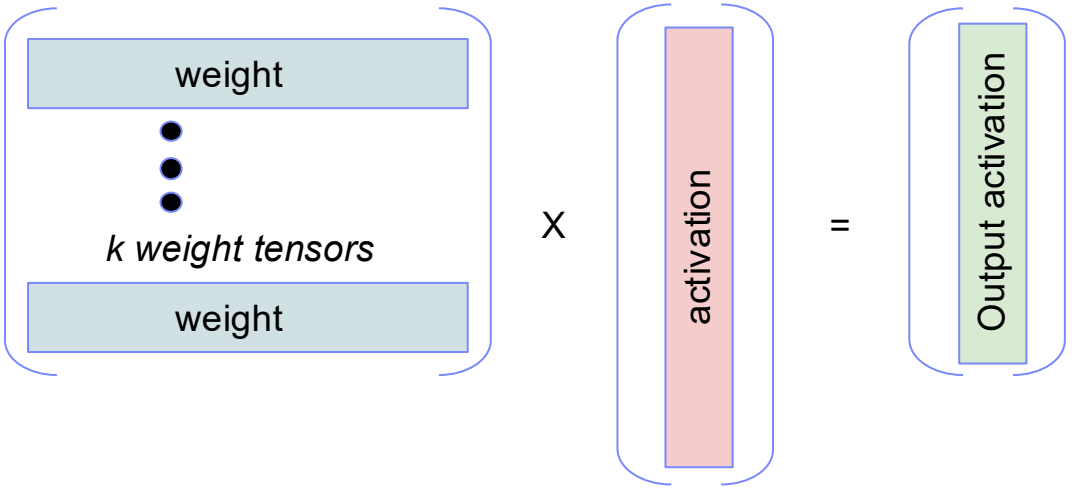
Linear or Fully-Connected

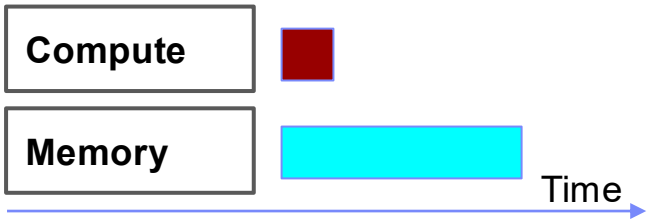


Linear or Fully-Connected



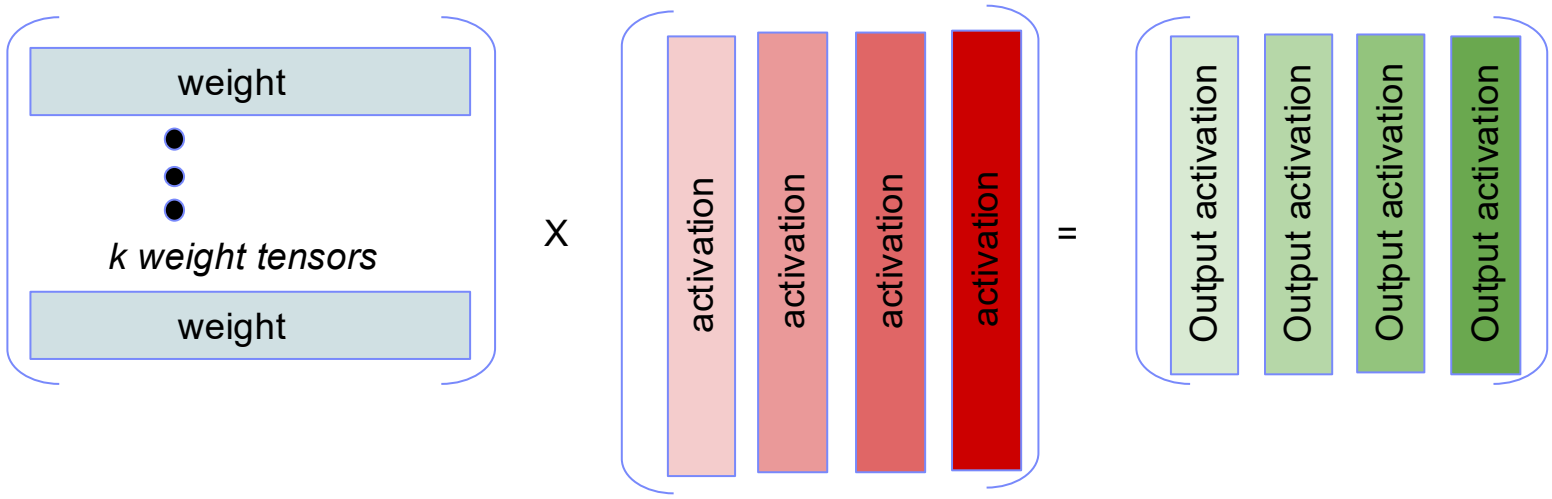
Linear Layers are Matrix Multiplications

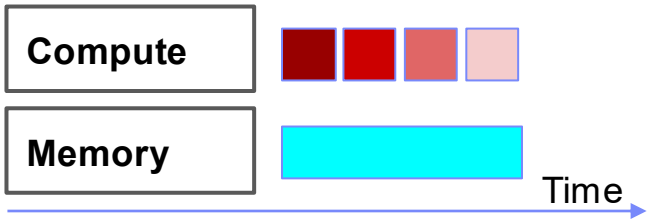




Memory-bound or compute bound?

Linear Layers are Matrix Multiplications: Batching





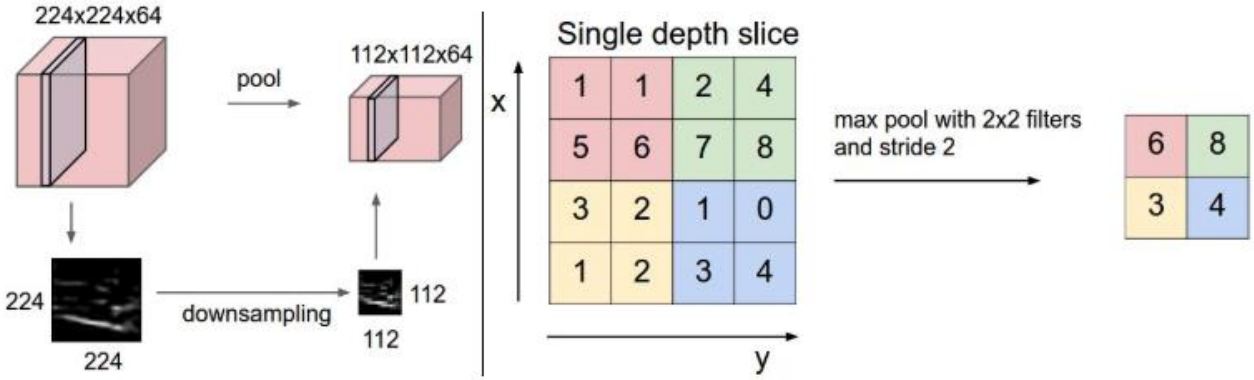
Memory-bound or compute bound?

Pooling, Normalization and Activation Functions

- **Typically computationally inexpensive (More details in the Book)**
- **Beware of pitfalls**
  - E.g.: Activation functions that cannot be computed in-place would need gradients to be computed before and after the activation function
  - Some activation functions have parameters (e.g.: PReLU)
  - Batchnorm becomes a simple scale + shift operation during inference
    - How slow is Batchnorm during training?
  - Pooling can reuse hardware for convolutions with a max/avg filter, but
  - A dedicated pooling unit will definitely be more efficient

Pooling Layer

- **Progressively reduce the spatial size of the representation to reduce the amount of parameters and computation in the network, and hence to also control overfitting.**
- **Parameters:**
  - Type: MAX and AVG
  - Pooling kernel size
  - Pooling stride

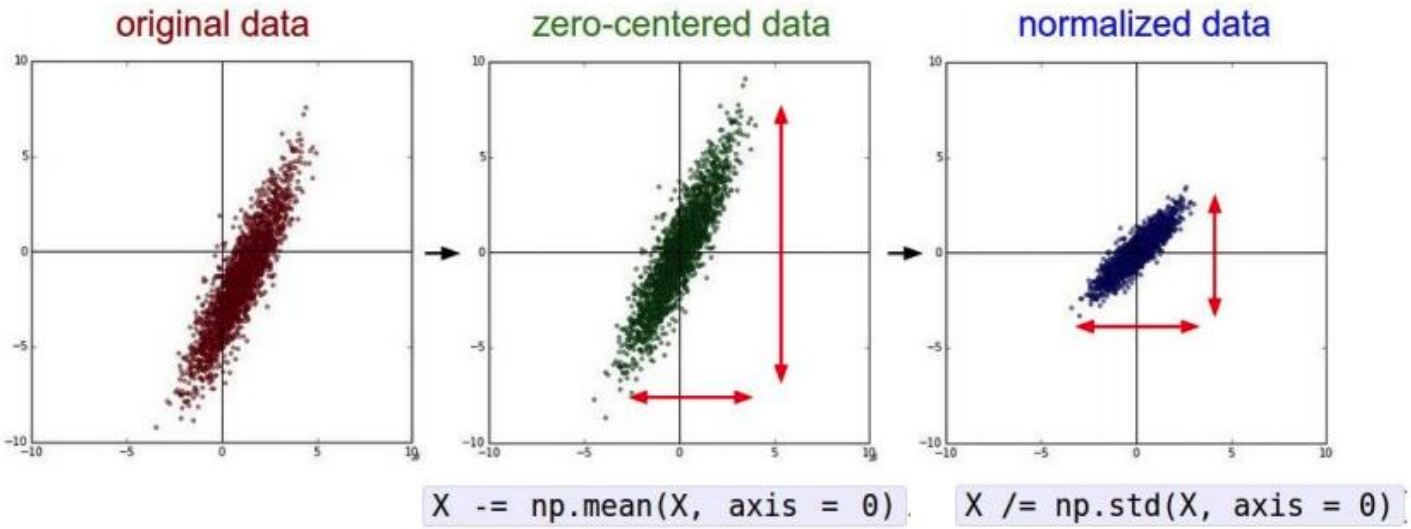


Pooling layer downsamples the volume spatially, independently in each depth slice of the input volume. Left: In this example, the input volume of size [224x224x64] is pooled with filter size 2, stride 2 into output volume of size [112x112x64]. Notice that the volume depth is preserved. Right: The most common downsampling operation is max, giving rise to max pooling, here shown with a stride of 2. That is, each max is taken over 4 numbers (little 2x2 square).

cs231n notes

BatchNorm Layer

- **Goal: make it easier to train by providing zero-mean, unit-variance activations.**
  - The training is complicated by the fact that the inputs to each layer are affected by the parameters of all preceding layers – so that small changes to the network parameters amplify as the network becomes deeper.



Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift

BatchNorm Layer

Training Time

Input: $x : N \times D$

Learnable scale and shift parameters:

$$\gamma, \beta : D$$

Learning $\gamma = \sigma$, $\beta = \mu$ will recover the identity function!

$$\mu_j = \frac{1}{N}\sum_{i=1}^{N} x_{i,j}$$ Per-channel mean, shape is D

$$\sigma_j^2 = \frac{1}{N}\sum_{i=1}^{N}(x_{i,j} - \mu_j)^2$$ Per-channel var, shape is D

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$ Normalized x, Shape is N x D

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$ Output, Shape is N x D

Test Time

$\mu_j =$ (Running) average of values seen during training — Per-channel mean, shape is D

$\sigma_j^2 =$ (Running) average of values seen during training — Per-channel var, shape is D

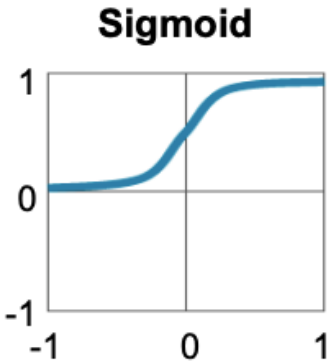
$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$ Normalized x, Shape is N x D

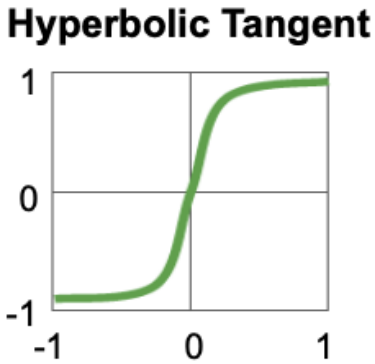
$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$ Output, Shape is N x D

DNN Non-Linearity

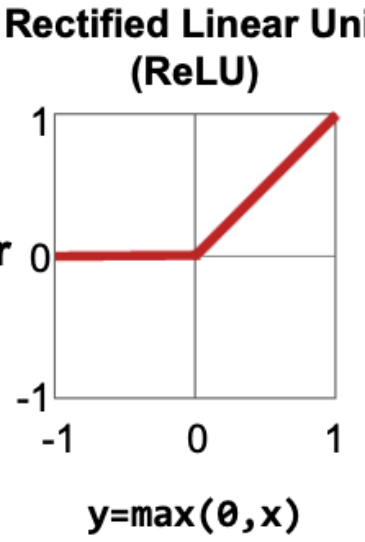
- **Rectified Linear Units (ReLU)**
  - $y(x) = \max\{0, x\}$
- **Benefits:**
  - Reduce the likelihood of the gradient vanishing problem.
  - Adding more sparsity/regularization
  - Easy to compute

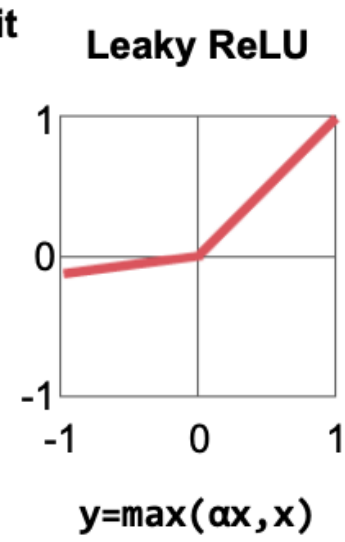
Traditional Non-Linear Activation Functions

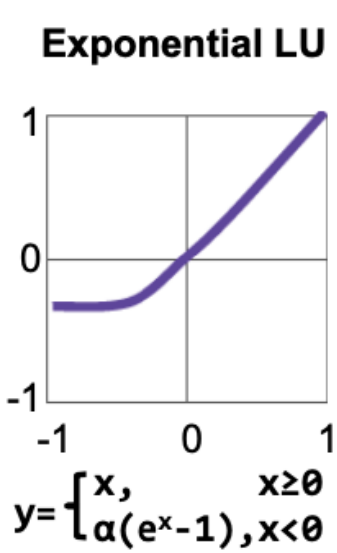




Modern Non-Linear Activation Functions

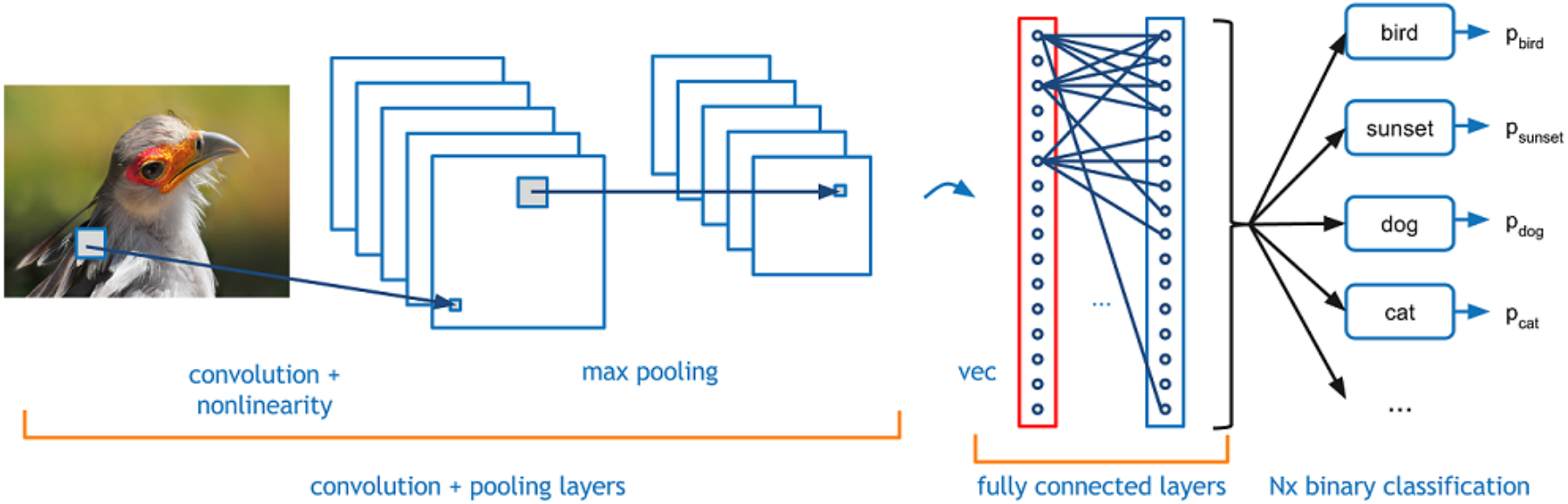






α = small const. (e.g. 0.1)

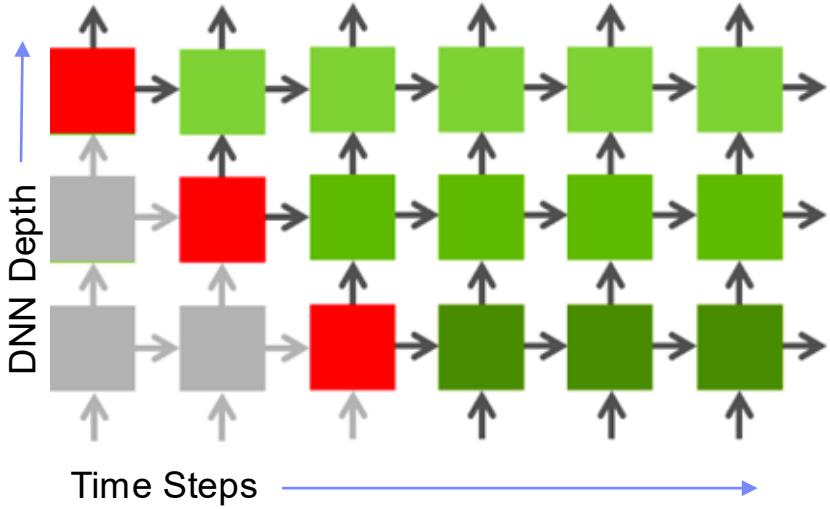
Convolutional Neural Nets

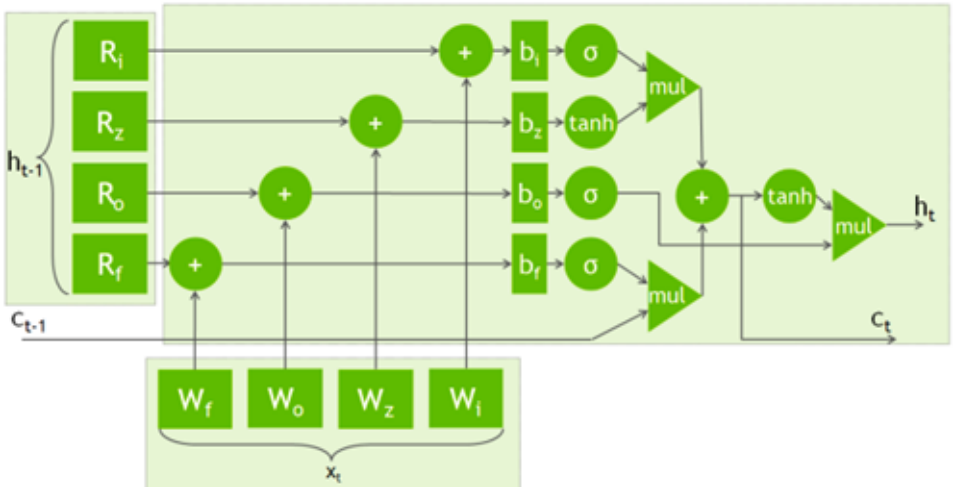


https://github.com/vdumoulin/conv_arithmetic

RNNs (LSTMs)

- **Data dependencies**
  - Within cell
  - Between cells

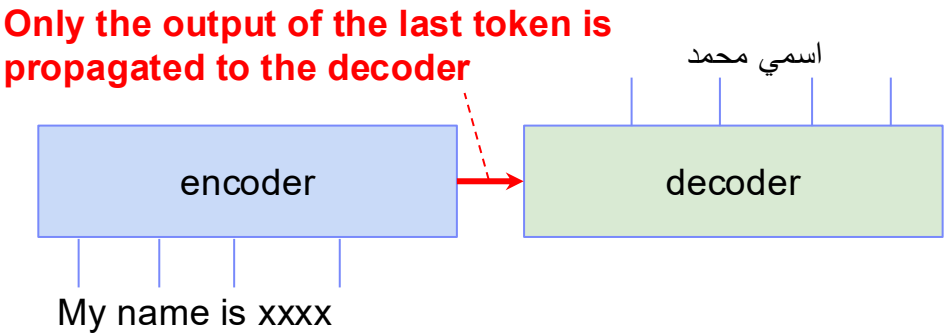




Source: Nice case study of optimizing LSTMs on Nvidia GPUs

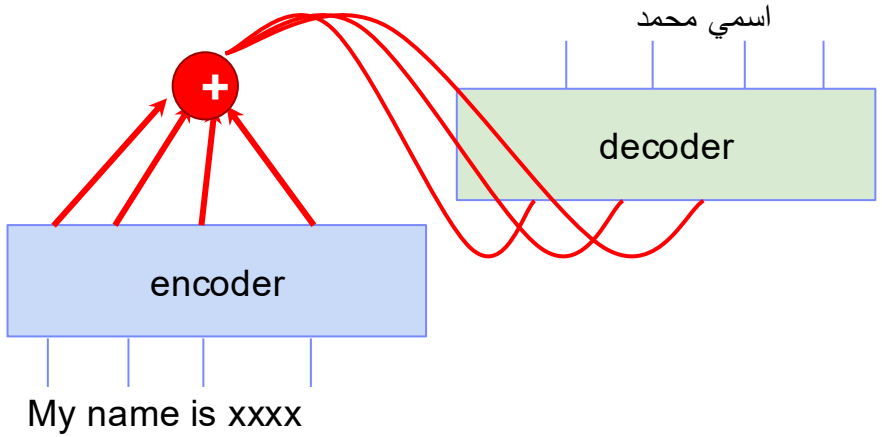
Attention (Transformers)

Encoder-Decoder Architecture



Attention

$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j.$$



Attention (Transformers)

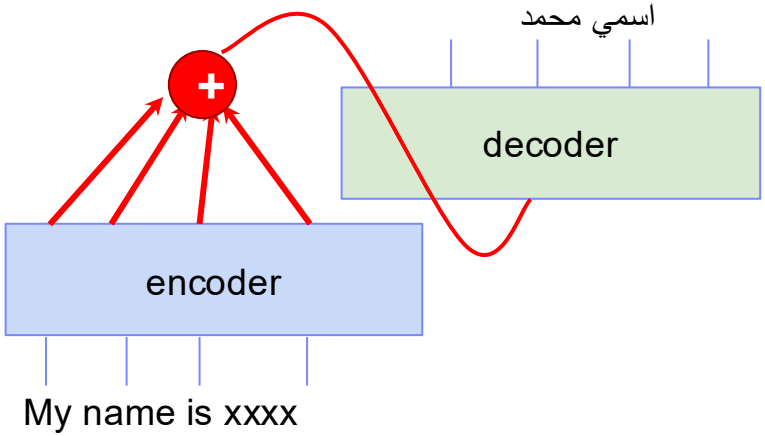
Attention

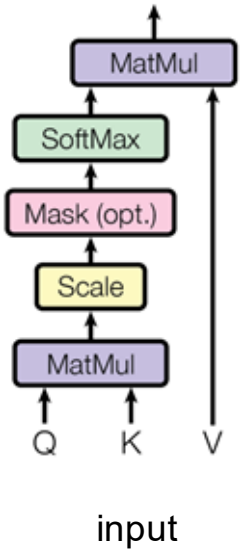
$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j.$$

Read this

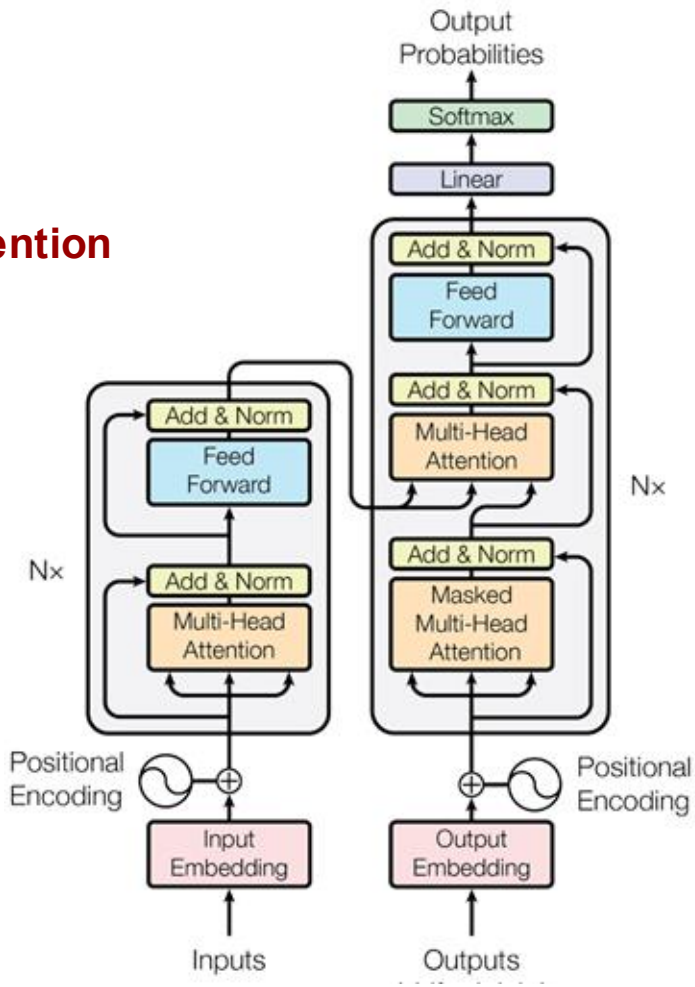
Scaled Dot Product Attention

$$\text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$





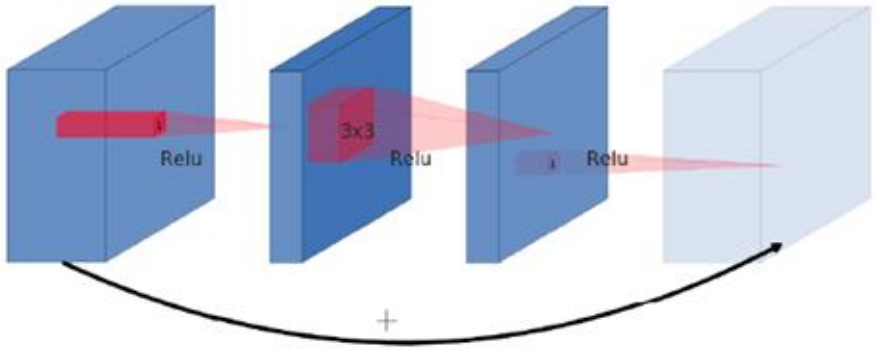
input



Source: attention is all you need

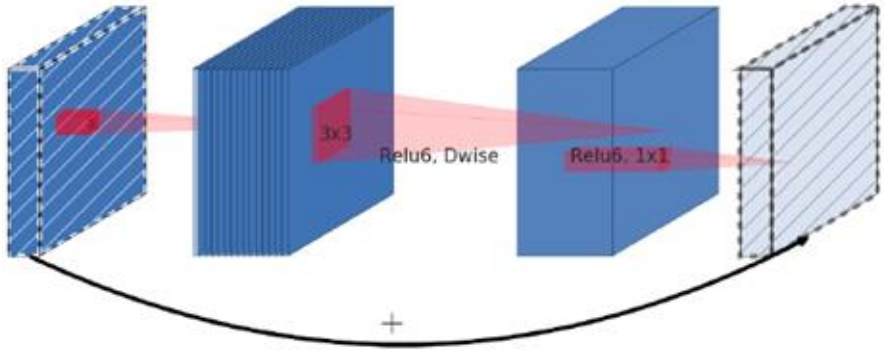
Compound CNN Layers

ResNet Bottleneck Block



Wide → Narrow → Wide

MobileNet Inverted Bottleneck Block



Narrow → Wide → Narrow

Fewer parameters!

Source: https://towardsdatascience.com/mobilenetv2-inverted-residuals-and-linear-bottlenecks-8a4362f4ffd5